

EEE 212 (January 2025)

Numerical Technique Laboratory

Final Project Report

Section: C1 Group: 06

Modeling the Spread of COVID-19 Using a Logistic Growth Model in MATLAB

Presentation Video: The final presentation for this project can be viewed at the following link: <https://youtu.be/kMos3fnVl4Q> or [Click Here](#)

Course Instructors:

Shafin-Bin-Hamid, Assistant Professor, EEE, BUET

Anudwaipon Antu, Part-Time Lecturer, EEE, BUET

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

"In signing this statement, we hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, we will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: *Sasshata Talukder*

Full Name: Sasshata Talukder

Student ID: 2206136

Signature: *S. M. Sindid Islam Mahodi*

Full Name: S. M. Sindid Islam Mahodi

Student ID: 2206147

Signature: *Rajib Khan*

Full Name: Rajib Khan

Student ID: 2206152

Signature: *Iftekhhar-E-Islam*

Full Name: Iftekhhar-E-Islam

Student ID: 2206153

Table of Contents

	Topic name	Page no.
1.	Abstract	03
2.	Introduction	03
3.	Design	04
3.1	Problem Formulation	04
3.2	Design Method	05
3.3	Simulation Model	06
3.4	Full Source Code	08
4.	Implementation	25
4.1	Description	25
5.	Design Analysis and Evaluation	26
5.1	Design Consideration	26
5.2	Investigations	27
5.2.1	Design of Experiment	27
5.2.2	Data Collection	27
5.2.3	Results	28
5.2.4	Conclusion on Data	31
5.3	Limitation of Tools	31
5.4	Sustainability Evaluation	32
6.	Reflection on Individual and Team Work	32
6.1	Individual Contribution of Each Member	32
6.2	Log Book of Project Implementation	33
7.	Communication to External Stakeholders	35
7.1	Executive Summary	35
7.2	User Manual	35
7.3	YouTube Link	38
8.	Project Management and Cost Analysis	38
9.	Future Work	39
9.1	Model Stability	39
9.2	Model Accuracy	39
10.	References	40
10.1	Ref from Websites	40

1. Abstract:

This project presents a MATLAB-based tool for modelling the cumulative spread of COVID-19 using a logistic growth model. Real-world case data is used to fit the model and estimate key parameters such as carrying capacity, growth rate, and inflection point. A graphical user interface (GUI) allows users to select countries, visualize trends, compare outbreaks, and perform advanced analytics including forecast and saturation date prediction. While the model assumes a single-wave outbreak and has limitations, it offers a simplified yet effective way to understand disease progression and explore epidemic trends interactively.

2. Introduction:

The COVID-19 pandemic has had a profound impact on global health, economies, and daily life. Accurately understanding how the disease spreads is critical for informed decision-making and effective public health responses. Mathematical modelling offers a powerful approach to forecast trends and analyze the dynamics of infectious diseases.

This project focuses on modelling the cumulative number of COVID-19 cases using the **logistic growth model**, which is well-suited for representing single-wave epidemic behaviour. The model provides three key parameters:

- **Carrying capacity (K):** the maximum expected cases,
- **Growth rate (r):** the speed of infection spread, and
- **Inflection point (t₀):** when the rate of new cases is highest.

To support data analysis and accessibility, we developed a **MATLAB-based graphical user interface (GUI)**. The GUI allows users to:

- Upload and visualize real-world, country-wise COVID-19 case data,
- Select one or two countries for comparison,
- Fit a logistic model using optimization methods like *lsqcurvefit*,
- Analyze residuals, forecast future trends, and evaluate model sensitivity.

The data used in this project was sourced from **Our World in Data**, offering reliable daily case counts across countries.

By combining mathematical modelling, data visualization, and interactive software design, this project provides a simplified yet effective tool for understanding epidemic growth. While the model assumes a single-wave outbreak and does not account for external interventions, it serves as a useful baseline for analysing case trends and informing future model enhancements.

3. Design:

3.1 Problem Formulation:

Analysing COVID-19 case data involves observing the cumulative infection trends and assessing how accurately these can be modelled using a logistic growth function. The process includes understanding the epidemic curve, preparing the data, applying the model, and interpreting the results.

Cumulative case data typically follows an **S-shaped (sigmoidal) curve**, especially in single-wave outbreaks. This aligns with logistic behaviour:

- **Initial phase:** rapid exponential growth,
- **Middle phase:** slowing spread due to interventions or immunity,
- **Final phase:** saturation as cases approach a limit (carrying capacity).

The dataset from **Our World in Data** includes daily case counts, dates, and country identifiers. Within the GUI, users select a country, and relevant date and case vectors are extracted. Data is then cleaned and normalized before fitting. Outliers or inconsistencies are handled to ensure reliable model performance.

The logistic model used is defined as:

$$N(t) = \frac{K}{1 + e^{-r(t-t_0)}}$$

Where:

- $N(t)$: cumulative number of cases at time t ,
- K : carrying capacity,
- r : growth rate,
- t_0 : inflection point (peak growth time).

The logistic curve fitting is done using MATLAB's `lsqcurvefit` function, which optimizes the parameters to minimize the mean squared error (MSE) between the model output and actual data.

Here

- Carrying Capacity (K): Indicates the estimated maximum number of cases the country may reach under current conditions.
- Growth Rate (r): Reflects how quickly the cases are increasing.
- Inflection Point (t_0): The date when the curve switches from accelerating to decelerating growth.

These parameters allow us to compare how different countries managed the spread and whether they are nearing saturation or still in a growth phase.

The GUI includes:

- Plots comparing actual vs. predicted data.
- Residual plots to observe fitting accuracy.
- Forecasted curves beyond the available data to check model reliability.

Through this analysis, we validate the logistic model's suitability for capturing the dynamics of COVID-19 spread, particularly for regions with a single wave or controlled outbreaks.

3.2 Design Method:

The design methodology of this project revolves around integrating a mathematical model (logistic growth) with an interactive graphical user interface (GUI) in MATLAB. The aim is to provide users with both analytical accuracy and ease of exploration through visual tools. The following steps outline the structured approach taken in the system design:

The logistic growth model is selected for its capability to simulate the cumulative case curve of COVID-19:

The logistic model used is defined as:

$$N(t) = \frac{K}{1 + e^{-r(t-t_0)}}$$

Data Integration and Curve Fitting

- The COVID-19 dataset is read from a .csv file using `readtable()`.
- Country names, dates, and case numbers are extracted.
- Data is filtered for the selected country using dropdown menus in the GUI.
- Missing or inconsistent values are handled through interpolation or exclusion.
- Initial guesses and bounds for the parameters are defined to improve fitting stability.
- The logistic function is implemented as an inline or anonymous function in the app.

GUI Interface Design

Using MATLAB App Designer:

- A responsive GUI was created with tabs and panels.
- Users can:
 - Select one or two countries for comparison.
 - View fitted curves and residuals.
 - Forecast future cases.
 - Explore advanced analytics (growth rate, saturation level, peak time).
 - Reset plots, zoom in, or save graphs.

Interface controls include:

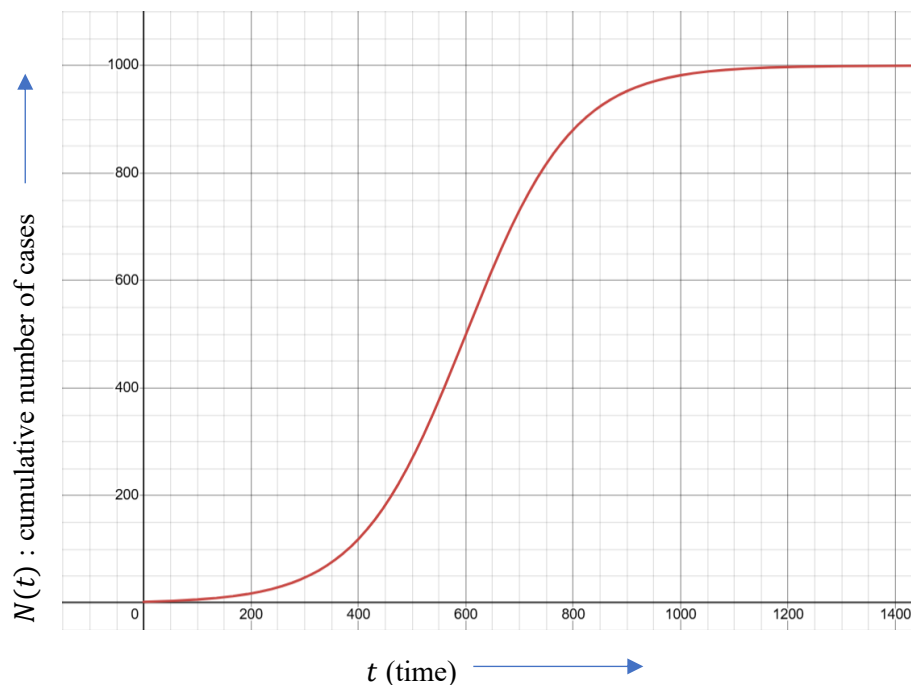
- Dropdown menus for country selection.
- Buttons for plotting, comparing, and resetting.
- Axes for displaying plots.
- Additional sliders for parameter tuning (if desired in future upgrades).

This methodical design approach ensures that the app not only performs accurate model fitting but also remains accessible, informative, and user-friendly for a broad audience.

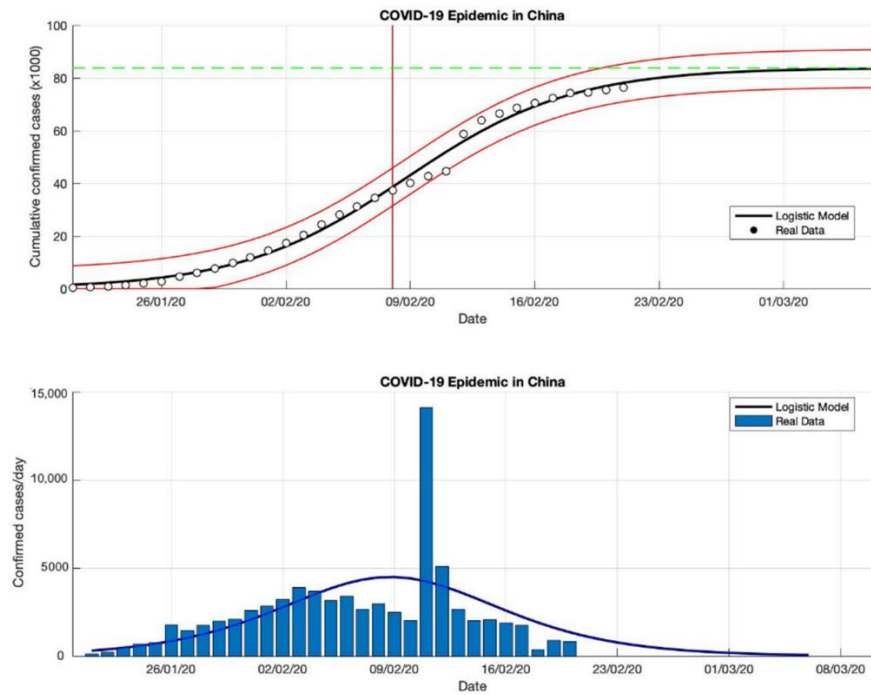
3.3 Simulation Model:

The simulation model uses the fitted logistic function to predict cumulative COVID-19 cases over time. By generating model outputs across a future time range, the system forecasts case trends, identifies saturation points, and visualizes epidemic dynamics through plots, aiding interpretation of disease progression for different countries.

Below here a sample graph of logistic model $N(t)$ vs t



This is the prediction of the Coronavirus Disease 2019 (COVID-19) Epidemic of a country (China) using Estimation of Parameters in the Logistic Growth Model



3.4 Full Source Code:

The complete MATLAB source code for the logistic model simulation and GUI interface is provided below for reference and reproducibility.

```
classdef COVID19_Logistic_GUI_App < matlab.apps.AppBase
```

```
% Properties that correspond to app components
properties (Access = public)
    UIFigure          matlab.ui.Figure
    UploadButton       matlab.ui.control.Button
    Country1DropDownLabel matlab.ui.control.Label
    Country1DropDown   matlab.ui.control.DropDown
    Country2DropDownLabel matlab.ui.control.Label
    Country2DropDown   matlab.ui.control.DropDown
    SwapButton         matlab.ui.control.Button

    % Country Search Components
    CountrySearchLabel matlab.ui.control.Label
    CountrySearchField  matlab.ui.control.EditField
    CountryListBox       matlab.ui.control.ListBox
```

```
    TabGroup           matlab.ui.container.TabGroup
    Country1Tab         matlab.ui.container.Tab
    Axes1_1             matlab.ui.control.UIAxes
    Axes1_2             matlab.ui.control.UIAxes
    Axes1_3             matlab.ui.control.UIAxes
    Axes1_4             matlab.ui.control.UIAxes
    K1Slider            matlab.ui.control.Slider
    K1SliderLabel       matlab.ui.control.Label
    K1ValueLabel        matlab.ui.control.Label
    r1Slider            matlab.ui.control.Slider
    r1SliderLabel       matlab.ui.control.Label
    r1ValueLabel        matlab.ui.control.Label
    t01Slider           matlab.ui.control.Slider
    t01SliderLabel      matlab.ui.control.Label
    t01ValueLabel       matlab.ui.control.Label
    FitModelButton      matlab.ui.control.Button
    ResetSlidersButton  matlab.ui.control.Button
    Export1Button       matlab.ui.control.Button
    Metrics1Label       matlab.ui.control.Label
```

```
    Country2Tab         matlab.ui.container.Tab
    Axes2_1             matlab.ui.control.UIAxes
    Axes2_2             matlab.ui.control.UIAxes
    Axes2_3             matlab.ui.control.UIAxes
    Axes2_4             matlab.ui.control.UIAxes
    K2Slider            matlab.ui.control.Slider
    K2SliderLabel       matlab.ui.control.Label
    K2ValueLabel        matlab.ui.control.Label
    r2Slider            matlab.ui.control.Slider
    r2SliderLabel       matlab.ui.control.Label
    r2ValueLabel        matlab.ui.control.Label
    t02Slider           matlab.ui.control.Slider
    t02SliderLabel      matlab.ui.control.Label
    t02ValueLabel       matlab.ui.control.Label
    FitModel2Button     matlab.ui.control.Button
    ResetSliders2Button matlab.ui.control.Button
    Export2Button       matlab.ui.control.Button
    Metrics2Label       matlab.ui.control.Label
```

```
    ComparisonTab       matlab.ui.container.Tab
    ComparisonAxes       matlab.ui.control.UIAxes
    ExportComparisonButton matlab.ui.control.Button
    ComparisonMetricsLabel matlab.ui.control.Label
```

```
    StatusLabel          matlab.ui.control.Label
    ProgressGauge         matlab.ui.control.LinearGauge
    AdvancedTab           matlab.ui.container.Tab
    AdvAnalysisDropdown   matlab.ui.control.DropDown
    AdvCountry1Dropdown   matlab.ui.control.DropDown
```

```
    AdvCountry2Dropdown   matlab.ui.control.DropDown
    AdvDateField          matlab.ui.control.EditField
    AdvRunButton          matlab.ui.control.Button
    AdvMetricsButton      matlab.ui.control.Button
    AdvThemeButton        matlab.ui.control.Button
    AdvAxes1              matlab.ui.control.UIAxes
    AdvAxes2              matlab.ui.control.UIAxes
    AdvAxes3              matlab.ui.control.UIAxes
    AdvAxes4              matlab.ui.control.UIAxes
    AdvResultLabel        matlab.ui.control.Label
    AdvAboutButton        matlab.ui.control.Button
    Report1Button          matlab.ui.control.Button
    Report2Button         matlab.ui.control.Button
    EventMarkerDate1      matlab.ui.control.DropDown
    EventMarkerLabel1     matlab.ui.control.EditField
    AddMarkerButton1      matlab.ui.control.Button
    MarkerListBox1        matlab.ui.control.ListBox
    RemoveMarkerButton1   matlab.ui.control.Button
    EventMarkerDate2      matlab.ui.control.DropDown
    EventMarkerLabel2     matlab.ui.control.EditField
    AddMarkerButton2      matlab.ui.control.Button
    MarkerListBox2        matlab.ui.control.ListBox
    RemoveMarkerButton2   matlab.ui.control.Button
end
```

```
properties (Access = private)
```

```
    DataTable
    CountryList
    Country1Data
    Country2Data
    Country1Model
    Country2Model
```

```
% Color scheme
bgColor = [0.15 0.18 0.22] % Darker blue-gray
panelColor = [0.20 0.23 0.28] % Panel background
accentColor = [0.26 0.56 0.85] % Modern blue accent
secondaryColor = [0.18 0.73 0.65] % Teal accent
textColor = [0.95 0.97 1] % Off-white text
labelColor = [0.8 0.85 0.9] % Light gray labels
errorColor = [0.9 0.3 0.3] % Red for errors
successColor = [0.3 0.8 0.4] % Green for success
isDarkMode = true
WorldData
WorldModel
light_bgColor = [1 1 1];
light_panelColor = [0.97 0.97 0.97];
light_accentColor = [0.12 0.34 0.85];
light_secondaryColor = [0.17 0.65 0.47];
light_textColor = [0.2 0.2 0.2];
light_labelColor = [0.25 0.25 0.25];
EventMarkers1 % Table for Country 1 event markers
EventMarkers2 % Table for Country 2 event markers
end
```

```
methods (Access = private)
```

```
function startupFcn(app)
    % Apply modern theme
    try
        applyModernTheme(app);
    catch
        % Continue if theme fails
    end
```

```
% Initialize GUI state
app.StatusLabel.Text = 'Ready - Please upload COVID-19 data file';
app.ProgressGauge.Value = 0;
```

```
% Disable controls until data is loaded
enableControls(app, false);
end
```

```
function applyModernTheme(app)
```



```

        if app.isDarkMode
            bg = app.bgColor; pn = app.panelColor; ac =
app.accentColor;
            sc = app.secondaryColor; tx = app.textColor; lb =
app.labelColor;
        else
            bg = app.light_bgColor; pn = app.light_panelColor; ac
= app.light_accentColor;
            sc = app.light_secondaryColor; tx = [0.1 0.1 0.1]; lb =
[0.2 0.2 0.2]; % darker for light mode
            buttonBG = [0.85 0.85 0.85];
            buttonFG = [0.1 0.1 0.1];
        end
        app.UIFigure.Color = bg;
        app.StatusLabel.FontColor = lb;
        app.StatusLabel.BackgroundColor = pn;
        app.Country1Tab.BackgroundColor = pn;
        app.Country2Tab.BackgroundColor = pn;
        app.ComparisonTab.BackgroundColor = pn;
        app.AdvancedTab.BackgroundColor = pn;
        bNames =
{'UploadButton','FitModelButton','FitModel2Button','AdvRunB
utton','AdvMetricsButton','AdvAboutButton','ResetSlidersButto
n','ResetSliders2Button'};
        for n = bNames
            if isprop(app, n{1})
                b = app.(n{1});
                if app.isDarkMode
                    b.BackgroundColor = ac; b.FontColor = tx;
                else
                    b.BackgroundColor = buttonBG; b.FontColor =
buttonFG;
                end
                b.FontWeight='bold'; b.FontSize=16;
                b.Position(2)=b.Position(2); b.Position(4)=30;
            end
        end
        sNames =
{'SwapButton','ResetSlidersButton','ResetSliders2Button','AdvT
hemeButton'};
        for n = sNames, s = app.(n{1}); s.BackgroundColor = sc;
s.FontColor = tx; s.FontSize=16; s.Position(2)=s.Position(2);
s.Position(4)=30; end
        exNames =
{'Export1Button','Export2Button','ExportComparisonButton'};
        for n = exNames, e = app.(n{1}); e.BackgroundColor =
[0.5 0.5 0.5]; e.FontColor = tx; e.FontSize=16;
e.Position(2)=e.Position(2); e.Position(4)=30; end
        ddNames =
{'Country1DropDown','Country2DropDown','AdvAnalysisDrop
down','AdvCountry1Dropdown','AdvCountry2Dropdown'};
        for n = ddNames, d = app.(n{1}); d.BackgroundColor =
pn; d.FontColor = tx; d.FontSize=15; end
        efNames = {'AdvDateField'};
        for n = efNames, f = app.(n{1}); f.BackgroundColor =
pn; f.FontColor = tx; f.FontSize=15; end
        if isprop(app, 'AdvResultLabel') &&
~isempty(app.AdvResultLabel)
            app.AdvResultLabel.FontSize = 15;
            app.AdvResultLabel.FontColor = tx;
        end
        % Set font color for parameter labels and metrics labels
        paramLabels =
{'K1SliderLabel','r1SliderLabel','t01SliderLabel','K2SliderLabel'
,'r2SliderLabel','t02SliderLabel'};
        for n = paramLabels
            if isprop(app, n{1})
                if app.isDarkMode
                    app.(n{1}).FontColor = [1 1 1];
                else
                    app.(n{1}).FontColor = [0.1 0.1 0.1];
                end
            end
        end
        % Set font color and size for metrics labels

```

```

metricsLabels = {'Metrics1Label','Metrics2Label'};
for n = metricsLabels
    if isprop(app, n{1})
        app.(n{1}).FontSize = 20;
        if app.isDarkMode
            app.(n{1}).FontColor = [1 1 1];
        else
            app.(n{1}).FontColor = [0.1 0.1 0.1];
        end
    end
end
% Set font color and size for comparison metrics label
if isprop(app, 'ComparisonMetricsLabel')
    app.ComparisonMetricsLabel.FontSize = 20;
    app.ComparisonMetricsLabel.FontName = 'Courier
New';
    if app.isDarkMode
        app.ComparisonMetricsLabel.FontColor = [1 1 1];
    else
        app.ComparisonMetricsLabel.FontColor = [0.1 0.1
0.1];
    end
end
% Set axes colors for dark/light mode
axesNames =
{'Axes1_1','Axes1_2','Axes1_3','Axes1_4','Axes2_1','Axes2_2','
Axes2_3','Axes2_4','ComparisonAxes','AdvAxes1','AdvAxes2','
AdvAxes3','AdvAxes4'};
for n = axesNames
    if isprop(app, n{1})
        ax = app.(n{1});
        if app.isDarkMode
            ax.Color = app.bgColor;
            ax.XColor = app.textColor;
            ax.YColor = app.textColor;
            ax.GridColor = [0.5 0.5 0.5];
        else
            ax.Color = [1 1 1];
            ax.XColor = [0.2 0.2 0.2];
            ax.YColor = [0.2 0.2 0.2];
            ax.GridColor = [0.7 0.7 0.7];
        end
    end
end
% Set slider colors for visibility
sliderNames =
{'K1Slider','r1Slider','t01Slider','K2Slider','r2Slider','t02Slider'};
for n = sliderNames
    if isprop(app, n{1})
        if app.isDarkMode
            app.(n{1}).FontColor = [1 1 1];
        else
            app.(n{1}).FontColor = [0.1 0.1 0.1];
        end
    end
end
function toggleTheme(app)
    app.isDarkMode = ~app.isDarkMode;
    applyModernTheme(app);
end

function enableControls(app, enable)
    % Enable/disable controls based on data availability
    controls = [app.Country1DropDown,
app.Country2DropDown, app.SwapButton, ...
app.FitModelButton, app.FitModel2Button]; %
Removed app.CountrySearchField
    for ctrl = controls
        ctrl.Enable = enable;
    end
end

function updateProgress(app, value, message)
    app.ProgressGauge.Value = value;

```

```

app.StatusLabel.Text = message;
app.StatusLabel.FontColor = app.labelColor;
drawnow;
end

function UploadButtonPushed(app, event)
    updateProgress(app, 10, 'Selecting data file...');

    [file, path] = uigetfile( {...
        '*.csv;*.xlsx;*.mat', 'Data Files (*.csv, *.xlsx, *.mat)';
        ...
        '*.csv', 'CSV Files (*.csv)'; ...
        '*.xlsx;*.xls', 'Excel Files (*.xlsx, *.xls)'; ...
        '*.mat', 'MAT Files (*.mat)'; ...
        '*.*', 'All Files (*.*)' }, ...
        'Select COVID-19 Data File');

    if isequal(file, 0)
        updateProgress(app, 0, 'File selection cancelled');
        return;
    end

    try
        updateProgress(app, 30, 'Loading and validating
data...');

        % Enhanced file loading with better format detection
        fullPath = fullfile(path, file);
        [~, ~, ext] = fileparts(file);

        switch lower(ext)
            case '.csv'
                app.DataTable = readtable(fullPath,
'VariableNamingRule', 'preserve');
            case {'*.xlsx', '*.xls'}
                app.DataTable = readtable(fullPath,
'VariableNamingRule', 'preserve');
            case '.mat'
                loadedData = load(fullPath);
                fields = fieldnames(loadedData);
                app.DataTable = loadedData.(fields{1});
            otherwise
                error('Unsupported file format: %s', ext);
        end

        % Validate data structure
        if height(app.DataTable) == 0
            error('Empty dataset loaded');
        end

        updateProgress(app, 60, 'Processing country list...');

        % Enhanced country extraction
        countryColumn = findColumn(app, app.DataTable, ...
{'location', 'country', 'Country_Region', 'Entity',
'nation', 'region'});

        if isempty(countryColumn)
            error('No country column found. Available
columns: %s', ...
                strjoin(app.DataTable.Properties.VariableNames,
', '));
        end

        app.CountryList =
sort(unique(app.DataTable.(countryColumn)));

        % Update all UI elements
        updateCountryDropdowns(app);

        updateProgress(app, 100, sprintf('Data loaded! %d
countries, %d records', ...
            length(app.CountryList), height(app.DataTable)));
        app.StatusLabel.FontColor = app.successColor;

```

```

        enableControls(app, true);

        % Auto-select first two countries if available
        if length(app.CountryList) >= 2
            app.Country1DropDown.Value =
app.CountryList{1};
            Country1DropDownValueChanged(app, event);
            app.Country2DropDown.Value =
app.CountryList{2};
            Country2DropDownValueChanged(app, event);
        end

        catch ME
            updateProgress(app, 0, ['Error: ' ME.message]);
            app.StatusLabel.FontColor = app.errorColor;
            showDetailedError(app, ME);
        end
    end

    function column = findColumn(~, dataTable,
possibleNames)
        % Find column with flexible name matching
        column = '';
        columnNames = dataTable.Properties.VariableNames;

        for i = 1:length(possibleNames)
            % Exact match first
            if any(strcmp(columnNames, possibleNames{i}))
                column = possibleNames{i};
                return;
            end
            % Case-insensitive match
            if any(strcmpi(columnNames, possibleNames{i}))
                idx = find(strcmpi(columnNames,
possibleNames{i}), 1);
                column = columnNames{idx};
                return;
            end
            % Partial match
            if any(contains(lower(columnNames),
lower(possibleNames{i})))
                idx = find(contains(lower(columnNames),
lower(possibleNames{i})), 1);
                column = columnNames{idx};
                return;
            end
        end
    end

    function updateCountryDropdowns(app)
        % Ensure CountryList is a row cell array of char vectors
        if isstring(app.CountryList)
            app.CountryList = cellstr(app.CountryList); % convert
string array to cell array of char
        end
        if iscolumn(app.CountryList)
            app.CountryList = app.CountryList'; % make it a row
        end
        app.Country1DropDown.Items = [{'Select Country 1'},
app.CountryList];
        app.Country2DropDown.Items = [{'Select Country 2'},
app.CountryList];
        if isprop(app, 'AdvCountry1Dropdown')
            app.AdvCountry1Dropdown.Items = [{'Select
Country'}, app.CountryList];
            app.AdvCountry2Dropdown.Items = [{'Select
Country (Only for comparison)'}, app.CountryList];
        end
    end

    function showDetailedError(app, ME)
        % Show detailed error information
        errorMsg = sprintf('Error: %s\n\nDetails:\n',
ME.message);
        for i = 1:length(ME.stack)

```

```

        errorMsg = sprintf('%sFile: %s\nFunction: %s\nLine: %d\n\n', ...
            errorMsg, ME.stack(i).file, ME.stack(i).name, ME.stack(i).line);
    end

    % Create error dialog
    uialert(app.UIFigure, errorMsg, 'Error Loading Data', 'Icon', 'error');
end

% Country Search Functions
function CountrySearchFieldValueChanged(app, event)
    searchText = app.CountrySearchField.Value;
    if isempty(searchText)
        app.CountryListBox.Items = app.CountryList;
    else
        % Filter countries alphabetically with smart matching
        matches =
app.CountryList(contains(lower(app.CountryList), lower(searchText)));
        app.CountryListBox.Items = sort(matches);
    end
end

function CountryListBoxValueChanged(app, event)
    selectedCountry = app.CountryListBox.Value;
    if ~isempty(selectedCountry) &&
iscell(selectedCountry)
        selectedCountry = selectedCountry{1}; % Take first if
multiple selected
    end

    if ~isempty(selectedCountry)
        % Auto-fill the appropriate dropdown
        if strcmp(app.Country1DropDown.Value, 'Select
Country 1') || ...
strcmp(app.Country1DropDown.Value,
selectedCountry)
            app.Country1DropDown.Value = selectedCountry;
            Country1DropDownValueChanged(app, event);
        else
            app.Country2DropDown.Value = selectedCountry;
            Country2DropDownValueChanged(app, event);
        end

        % Clear search field
        app.CountrySearchField.Value = "";
        app.CountryListBox.Items = app.CountryList;
    end
end

function Country1DropDownValueChanged(app, event)
    country = app.Country1DropDown.Value;
    if strcmp(country, 'Select Country 1')
        return;
    end
    % Warn if same country is selected in both dropdowns
    if strcmp(app.Country2DropDown.Value, country)
        uialert(app.UIFigure, 'You have selected the same
country for both Country 1 and Country 2. Please fit both
models to ensure comparison is valid.', 'Same Country
Selected');
    end
    try
        updateProgress(app, 20, ['Loading data for ' country
'...']);
        % Extract data for country 1
        app.Country1Data = extractCountryData(app,
country);
        if isempty(app.Country1Data)
            error('No valid data found for %s', country);
        end
        updateSliderRanges(app, 1, app.Country1Data);
        plotCountryData(app, 1, app.Country1Data);
    end
end

```

```

% --- FIX: Clear previous model's plots and metrics --
-
    cla(app.Axes1_2); title(app.Axes1_2, 'Logistic Model
Fit');
    cla(app.Axes1_3); title(app.Axes1_3, 'Residuals
Analysis');
    cla(app.Axes1_4); title(app.Axes1_4, 'Future
Predictions');
    app.Metrics1Label.Text = 'Model metrics will appear
here...';

    updateEventMarkerDates(app, 1, app.Country1Data);
    updateProgress(app, 100, [' Country 1 data loaded: '
country]);
    app.StatusLabel.FontColor = app.successColor;
    % Reset model for Country 1 (do not copy from
Country 2)
    app.Country1Model = [];
    app.K1ValueLabel.Text =
num2str(app.K1Slider.Value);
    app.r1ValueLabel.Text =
num2str(app.r1Slider.Value);
    app.t01ValueLabel.Text =
num2str(app.t01Slider.Value);
    catch ME
        updateProgress(app, 0, [' Error processing ' country ': '
ME.message]);
        app.StatusLabel.FontColor = app.errorColor;
    end
end

function Country2DropDownValueChanged(app, event)
    country = app.Country2DropDown.Value;
    if strcmp(country, 'Select Country 2')
        return;
    end
    % Warn if same country is selected in both dropdowns
    if strcmp(app.Country1DropDown.Value, country)
        uialert(app.UIFigure, 'You have selected the same
country for both Country 1 and Country 2. Please fit both
models to ensure comparison is valid.', 'Same Country
Selected');
    end
    try
        updateProgress(app, 20, ['Loading data for ' country
'...']);
        % Extract data for country 2
        app.Country2Data = extractCountryData(app,
country);
        if isempty(app.Country2Data)
            error('No valid data found for %s', country);
        end
        updateSliderRanges(app, 2, app.Country2Data);
        plotCountryData(app, 2, app.Country2Data);
    end

    % --- FIX: Clear previous model's plots and metrics --
-
    cla(app.Axes2_2); title(app.Axes2_2, 'Logistic Model
Fit');
    cla(app.Axes2_3); title(app.Axes2_3, 'Residuals
Analysis');
    cla(app.Axes2_4); title(app.Axes2_4, 'Future
Predictions');
    app.Metrics2Label.Text = 'Model metrics will appear
here...';

    updateEventMarkerDates(app, 2, app.Country2Data);
    updateProgress(app, 100, [' Country 2 data loaded: '
country]);
    app.StatusLabel.FontColor = app.successColor;
    % Reset model for Country 2 (do not copy from
Country 1)
    app.Country2Model = [];
    app.K2ValueLabel.Text =

```

```

num2str(app.K2Slider.Value);
    app.r2ValueLabel.Text =
num2str(app.r2Slider.Value);
    app.t02ValueLabel.Text =
num2str(app.t02Slider.Value);
    catch ME
        updateProgress(app, 0, [' Error processing ' country ': '
ME.message]);
        app.StatusLabel.FontColor = app.errorColor;
    end
end

function data = extractCountryData(app, country)
    try
        % Find relevant columns
        columnMappings = struct(...
            'country', {'location', 'country', 'Country_Region',
'Entity', 'nation', 'region'}}, ...
            'date', {'date', 'Date', 'dateRep', 'time', 'Day',
'Date_reported'}}, ...
            'cases', {'total_cases', 'Cases', 'cases',
'Cumulative_cases', 'cumulative_cases', 'Total_cases'}} ...
        );

        countryCol = findColumn(app, app.DataTable,
columnMappings.country);
        dateCol = findColumn(app, app.DataTable,
columnMappings.date);
        casesCol = findColumn(app, app.DataTable,
columnMappings.cases);

        if isempty(countryCol) || isempty(dateCol) ||
isempty(casesCol)
            error('Required columns not found. Available: %s',
...
            strjoin(app.DataTable.Properties.VariableNames,
', '));
        end

        % Filter data for the specified country
        countryRows = strcmp(app.DataTable.(countryCol),
country);
        filteredData = app.DataTable(countryRows, :);

        if height(filteredData) == 0
            error('No data found for country: %s', country);
        end

        % Extract and process data
        dates = filteredData.(dateCol);
        cases = filteredData.(casesCol);

        % Handle different date formats
        if ~isdatetime(dates)
            if isnumeric(dates)
                dates = datetime(2020, 1, 1) + days(dates - 1);
            else
                dates = datetime(dates, 'InputFormat', 'auto');
            end
        end

        % Ensure cases are numeric
        if iscell(cases)
            cases = str2double(cases);
        elseif isstring(cases)
            cases = double(cases);
        end

        % Sort by date and clean data
        [dates, sortIdx] = sort(dates);
        cases = cases(sortIdx);

        % Remove invalid data points
        validIdx = ~isnan(cases) & cases >= 0;
        dates = dates(validIdx);

```

```

        cases = cases(validIdx);

        if isempty(dates)
            error('No valid data points found after cleaning');
        end

        % Calculate time in days
        timeDays = days(dates - dates(1));

        % Create structured output
        data = table(dates, cases, timeDays, ...
            'VariableNames', {'Date', 'Cases', 'TimeDays'});

        catch ME
            data = table();
            rethrow(ME);
        end

        function updateSliderRanges(app, countryNum, data)
            if isempty(data) || height(data) == 0
                return;
            end

            maxCases = max(data.Cases);
            maxTime = max(data.TimeDays);

            if countryNum == 1
                % Update Country 1 sliders
                app.K1Slider.Limits = [maxCases*0.5, maxCases*5];
                app.K1Slider.Value = maxCases * 1.2;
                app.K1ValueLabel.Text = sprintf('%0f',
app.K1Slider.Value);

                app.r1Slider.Limits = [0.01, 1.0];
                app.r1Slider.Value = 0.1;
                app.r1ValueLabel.Text = sprintf('%0.3f',
app.r1Slider.Value);

                app.t01Slider.Limits = [1, maxTime];
                app.t01Slider.Value = maxTime / 2;
                app.t01ValueLabel.Text = sprintf('%0f',
app.t01Slider.Value);
            else
                % Update Country 2 sliders
                app.K2Slider.Limits = [maxCases*0.5, maxCases*5];
                app.K2Slider.Value = maxCases * 1.2;
                app.K2ValueLabel.Text = sprintf('%0f',
app.K2Slider.Value);

                app.r2Slider.Limits = [0.01, 1.0];
                app.r2Slider.Value = 0.1;
                app.r2ValueLabel.Text = sprintf('%0.3f',
app.r2Slider.Value);

                app.t02Slider.Limits = [1, maxTime];
                app.t02Slider.Value = maxTime / 2;
                app.t02ValueLabel.Text = sprintf('%0f',
app.t02Slider.Value);
            end
        end

        function plotCountryData(app, countryNum, data)
            if isempty(data) || height(data) == 0
                return;
            end
            if countryNum == 1
                ax = app.Axes1_1;
                country = app.Country1DropDown.Value;
                markers = app.EventMarkers1;
            else
                ax = app.Axes2_1;
                country = app.Country2DropDown.Value;
                markers = app.EventMarkers2;
            end

```

```

cla(ax);
hold(ax, 'on'); % Hold on to add markers later
if app.isDarkMode
    plot(ax, data.Date, data.Cases, 'o-', 'Color', [0 1 1], ...
        'LineWidth', 2, 'MarkerSize', 4);
    title(ax, [country ' - Raw Data'], 'Color', [1 1 1]);
    xlabel(ax, 'Date', 'Color', [1 1 1]);
    ylabel(ax, 'Cumulative Cases', 'Color', [1 1 1]);
    ax.XColor = [1 1 1]; ax.YColor = [1 1 1];
else
    plot(ax, data.Date, data.Cases, 'o-', 'Color', [0 0 0], ...
        'LineWidth', 2, 'MarkerSize', 4);
    title(ax, [country ' - Raw Data'], 'Color', [0 0 0]);
    xlabel(ax, 'Date', 'Color', [0.1 0.1 0.1]);
    ylabel(ax, 'Cumulative Cases', 'Color', [0.1 0.1 0.1]);
    ax.XColor = [0.1 0.1 0.1]; ax.YColor = [0.1 0.1 0.1];
end
% Plot event markers
if ~isempty(markers) && istable(markers)
    for i = 1:height(markers)
        x = markers.Date(i);
        yl = ylim(ax);
        line(ax, [x x], yl, 'Color', [1 0.5 0], 'LineWidth', 2,
            'LineStyle', '--');
        text(ax, x, yl(2), ['\leftarrow ' markers.Label{i}],
            'Color', [1 0.5 0], 'FontWeight', 'bold', 'VerticalAlignment', 'top',
            'FontSize', 10);
    end
end
hold(ax, 'off');
end

% COMPLETE MODEL FITTING FOR COUNTRY 1
function FitModelButtonPushed(app, event)
    if isempty(app.Country1Data)
        app.StatusLabel.Text = ' Please select Country 1 first';
        app.StatusLabel.FontColor = app.errorColor;
        return;
    end

    updateProgress(app, 20, 'Fitting logistic model for
Country 1...');

    try
        % Prepare data for fitting
        data = app.Country1Data;
        t_data = data.TimeDays;
        N_data = data.Cases;

        % Remove zero cases for better fitting
        nonzero_idx = N_data > 0;
        t_fit = t_data(nonzero_idx);
        N_fit = N_data(nonzero_idx);

        if isempty(t_fit)
            error('No positive case data available for fitting');
        end

        % Define logistic model
        logistic_model = @(params, t) params(1) ./ (1 + exp(-
params(2) * (t - params(3))));

        % Initial parameter estimates
        K_init = max(N_fit) * 1.5;
        r_init = 0.1;
        t0_init = t_fit(round(end/2));
        initial_params = [K_init, r_init, t0_init];

        % Parameter bounds
        lower_bounds = [max(N_fit), 0.01, 0];
        upper_bounds = [max(N_fit)*10, 1, max(t_fit)*1.5];

        updateProgress(app, 50, 'Running optimization...');
    end
end

```

```

% Curve fitting
options = optimoptions('lsqcurvefit', 'Display', 'off',
'MaxIterations', 1000);
[fitted_params, ~, ~, exitflag] = lsqcurvefit(...
    logistic_model, initial_params, t_fit, N_fit,
    lower_bounds, upper_bounds, options);

if exitflag <= 0
    % Fallback optimization
    objective_function = @(params)
sum((logistic_model(params, t_fit) - N_fit).^2);
    fitted_params = fminsearch(objective_function,
    initial_params);
end

K_fitted = fitted_params(1);
r_fitted = fitted_params(2);
t0_fitted = fitted_params(3);

% Update sliders
app.K1Slider.Value = K_fitted;
app.r1Slider.Value = r_fitted;
app.t01Slider.Value = t0_fitted;
app.K1ValueLabel.Text = sprintf('%0f', K_fitted);
app.r1ValueLabel.Text = sprintf('%0.3f', r_fitted);
app.t01ValueLabel.Text = sprintf('%0f', t0_fitted);

updateProgress(app, 80, 'Generating plots and
metrics...');

% Generate predictions and plot results
plotModelResults(app, 1, data, fitted_params);

% Store model results
app.Country1Model = struct('params', fitted_params,
'data', data);

updateProgress(app, 100, ' Model fitted successfully
for Country 1');
app.StatusLabel.FontColor = app.successColor;

if ~isempty(app.Country2Model)
    updateComparisonPlot(app);
end
catch ME
    updateProgress(app, 0, [' Error fitting model: '
ME.message]);
    app.StatusLabel.FontColor = app.errorColor;
end
end

% COMPLETE MODEL FITTING FOR COUNTRY 2
(FIXED)
function FitModel2ButtonPushed(app, event)
    if isempty(app.Country2Data)
        app.StatusLabel.Text = ' Please select Country 2 first';
        app.StatusLabel.FontColor = app.errorColor;
        return;
    end

    updateProgress(app, 20, 'Fitting logistic model for
Country 2...');

    try
        % Prepare data for fitting
        data = app.Country2Data;
        t_data = data.TimeDays;
        N_data = data.Cases;

        % Remove zero cases for better fitting
        nonzero_idx = N_data > 0;
        t_fit = t_data(nonzero_idx);
        N_fit = N_data(nonzero_idx);
    end
end

```

```

if isempty(t_fit)
    error('No positive case data available for fitting');
end

% Define logistic model
logistic_model = @(params, t) params(1) ./ (1 + exp(-
params(2) * (t - params(3))));

% Initial parameter estimates
K_init = max(N_fit) * 1.5;
r_init = 0.1;
t0_init = t_fit(round(end/2));
initial_params = [K_init, r_init, t0_init];

% Parameter bounds
lower_bounds = [max(N_fit), 0.01, 0];
upper_bounds = [max(N_fit)*10, 1, max(t_fit)*1.5];

updateProgress(app, 50, 'Running optimization...');

% Curve fitting
options = optimoptions('lsqcurvefit', 'Display', 'off',
'MaxIterations', 1000);
[fitted_params, ~, ~, exitflag] = lsqcurvefit(...
    logistic_model, initial_params, t_fit, N_fit,
lower_bounds, upper_bounds, options);

if exitflag <= 0
    % Fallback optimization
    objective_function = @(params)
sum((logistic_model(params, t_fit) - N_fit).^2);
    fitted_params = fminsearch(objective_function,
initial_params);
end

K_fitted = fitted_params(1);
r_fitted = fitted_params(2);
t0_fitted = fitted_params(3);

% Update sliders
app.K2Slider.Value = K_fitted;
app.r2Slider.Value = r_fitted;
app.t02Slider.Value = t0_fitted;
app.K2ValueLabel.Text = sprintf('%0f', K_fitted);
app.r2ValueLabel.Text = sprintf('%0.3f', r_fitted);
app.t02ValueLabel.Text = sprintf('%0f', t0_fitted);

updateProgress(app, 80, 'Generating plots and
metrics...');

% Generate predictions and plot results
plotModelResults(app, 2, data, fitted_params);

% Store model results
app.Country2Model = struct('params', fitted_params,
'data', data);

updateProgress(app, 100, ' Model fitted successfully
for Country 2');
app.StatusLabel.FontColor = app.successColor;

% Update comparison if both models exist
if ~isempty(app.Country1Model)
    updateComparisonPlot(app);
end

catch ME
    updateProgress(app, 0, [' Error fitting model: '
ME.message]);
    app.StatusLabel.FontColor = app.errorColor;
end
end

function plotModelResults(app, countryNum, data,
fitted_params)

```

```

    logistic_model = @(params, t) params(1) ./ (1 + exp(-
params(2) * (t - params(3))));
    t_data = data.TimeDays;
    N_data = data.Cases;
    N_predicted = logistic_model(fitted_params, t_data);
    residuals = N_data - N_predicted;
    rmse = sqrt(mean(residuals.^2));
    r_squared = 1 - sum(residuals.^2) / sum((N_data -
mean(N_data)).^2);
    if countryNum == 1
        axes_list = {app.Axes1_2, app.Axes1_3,
app.Axes1_4};
        metricsLabel = app.Metrics1Label;
        country = app.Country1DropDown.Value;
    else
        axes_list = {app.Axes2_2, app.Axes2_3,
app.Axes2_4};
        metricsLabel = app.Metrics2Label;
        country = app.Country2DropDown.Value;
    end
    % Plot 1: Model Fit
    cla(axes_list{1});
    hold(axes_list{1}, 'on');
    if app.isDarkMode
        plot(axes_list{1}, data.Date, N_data, 'o', 'Color', [0 1
1], ... % cyan
            'MarkerSize', 4, 'DisplayName', 'Actual Data');
        plot(axes_list{1}, data.Date, N_predicted, '-', 'Color',
[1 0 1], ... % magenta
            'LineWidth', 2, 'DisplayName', 'Model Fit');
        title(axes_list{1}, 'Logistic Model Fit', 'Color', [1 1
1]);
        axes_list{1}.XColor = [1 1 1]; axes_list{1}.YColor =
[1 1 1];
        lgd = legend(axes_list{1}, 'Location', 'southeast');
        lgd.TextColor = [1 1 1]; lgd.Color = [0.15 0.18 0.22];
    else
        plot(axes_list{1}, data.Date, N_data, 'o', 'Color', [0 0
1], ... % blue
            'MarkerSize', 4, 'DisplayName', 'Actual Data');
        plot(axes_list{1}, data.Date, N_predicted, '-', 'Color',
[1 0 0], ... % red
            'LineWidth', 2, 'DisplayName', 'Model Fit');
        title(axes_list{1}, 'Logistic Model Fit', 'Color', [0 0
0]);
        axes_list{1}.XColor = [0.1 0.1 0.1];
        axes_list{1}.YColor = [0.1 0.1 0.1];
        lgd = legend(axes_list{1}, 'Location', 'southeast');
        lgd.TextColor = [0.1 0.1 0.1]; lgd.Color = [1 1 1];
    end
    grid(axes_list{1}, 'on');
    hold(axes_list{1}, 'off');
    % Plot 2: Residuals
    cla(axes_list{2});
    if app.isDarkMode
        plot(axes_list{2}, data.Date, residuals, 'o-', 'Color', [1
0.5 0], 'MarkerSize', 3); % orange
        title(axes_list{2}, 'Residuals Analysis', 'Color', [1 1
1]);
        axes_list{2}.XColor = [1 1 1]; axes_list{2}.YColor =
[1 1 1];
    else
        plot(axes_list{2}, data.Date, residuals, 'o-', 'Color',
[0.5 0 0.5], 'MarkerSize', 3); % purple
        title(axes_list{2}, 'Residuals Analysis', 'Color', [0 0
0]);
        axes_list{2}.XColor = [0.1 0.1 0.1];
        axes_list{2}.YColor = [0.1 0.1 0.1];
    end
    hold(axes_list{2}, 'on');
    yline(axes_list{2}, 0, '--', 'Color', app.labelColor);
    hold(axes_list{2}, 'off');
    grid(axes_list{2}, 'on');
    % Plot 3: Future Predictions
    t_future = linspace(0, max(t_data) * 1.5, 500);

```

```

N_future = logistic_model(fitted_params, t_future);
date_future = data.Date(1) + days(t_future);
cla(axes_list{3});
hold(axes_list{3}, 'on');
if app.isDarkMode
    plot(axes_list{3}, data.Date, N_data, 'o', 'Color', [0 1
1], ... % cyan
        'MarkerSize', 4, 'DisplayName', 'Actual Data');
    plot(axes_list{3}, date_future, N_future, '-', 'Color', [1
0 1], ... % magenta
        'LineWidth', 2, 'DisplayName', 'Prediction');
    title(axes_list{3}, 'Future Predictions', 'Color', [1 1
1]);
    axes_list{3}.XColor = [1 1 1]; axes_list{3}.YColor =
[1 1 1];
    lgd3 = legend(axes_list{3}, 'Location', 'southeast');
    lgd3.TextColor = [1 1 1]; lgd3.Color = [0.15 0.18
0.22];
else
    plot(axes_list{3}, data.Date, N_data, 'o', 'Color', [0 0
1], ... % blue
        'MarkerSize', 4, 'DisplayName', 'Actual Data');
    plot(axes_list{3}, date_future, N_future, '-', 'Color', [0
0.5 0], ... % green
        'LineWidth', 2, 'DisplayName', 'Prediction');
    title(axes_list{3}, 'Future Predictions', 'Color', [0 0
0]);
    axes_list{3}.XColor = [0.1 0.1 0.1];
    axes_list{3}.YColor = [0.1 0.1 0.1];
    lgd3 = legend(axes_list{3}, 'Location', 'southeast');
    lgd3.TextColor = [0.1 0.1 0.1]; lgd3.Color = [1 1 1];
end
grid(axes_list{3}, 'on');
hold(axes_list{3}, 'off');
metricsText = sprintf([...
    'Fitted Parameters:\n' ...
    'K: %.0f\n' ...
    'r: %.4f\n' ...
    'to: %.1f days\n\n' ...
    'Model Quality:\n' ...
    'R²: %.4f\n' ...
    'RMSE: %.2f', ...
    fitted_params(1), fitted_params(2), fitted_params(3),
r_squared, rmse);
metricsLabel.Text = metricsText;
end

function SwapButtonPushed(app, event)
    % Swap countries
    c1 = app.Country1DropDown.Value;
    c2 = app.Country2DropDown.Value;

    app.Country1DropDown.Value = c2;
    app.Country2DropDown.Value = c1;

    % Swap data
    tempData = app.Country1Data;
    app.Country1Data = app.Country2Data;
    app.Country2Data = tempData;

    % Swap models
    tempModel = app.Country1Model;
    app.Country1Model = app.Country2Model;
    app.Country2Model = tempModel;

    % Refresh displays
    if ~isempty(app.Country1Data)
        updateSliderRanges(app, 1, app.Country1Data);
        plotCountryData(app, 1, app.Country1Data);
    end

    if ~isempty(app.Country2Data)
        updateSliderRanges(app, 2, app.Country2Data);
        plotCountryData(app, 2, app.Country2Data);
    end
end

```

```

        updateProgress(app, 100, 'Countries swapped
successfully');
        app.StatusLabel.FontColor = app.successColor;
    end

    % Real-time slider updates
    function K1SliderValueChanged(app, event)
        app.K1ValueLabel.Text = sprintf('%0f',
app.K1Slider.Value);
        updateModelPlot(app, 1);
    end

    function r1SliderValueChanged(app, event)
        app.r1ValueLabel.Text = sprintf('%0.3f',
app.r1Slider.Value);
        updateModelPlot(app, 1);
    end

    function t01SliderValueChanged(app, event)
        app.t01ValueLabel.Text = sprintf('%0f',
app.t01Slider.Value);
        updateModelPlot(app, 1);
    end

    function K2SliderValueChanged(app, event)
        app.K2ValueLabel.Text = sprintf('%0f',
app.K2Slider.Value);
        updateModelPlot(app, 2);
    end

    function r2SliderValueChanged(app, event)
        app.r2ValueLabel.Text = sprintf('%0.3f',
app.r2Slider.Value);
        updateModelPlot(app, 2);
    end

    function t02SliderValueChanged(app, event)
        app.t02ValueLabel.Text = sprintf('%0f',
app.t02Slider.Value);
        updateModelPlot(app, 2);
    end

    function updateModelPlot(app, countryNum)
        % Update plot with current slider values
        if countryNum == 1 && ~isempty(app.Country1Data)
            params = [app.K1Slider.Value, app.r1Slider.Value,
app.t01Slider.Value];
            plotModelResults(app, 1, app.Country1Data, params);
        elseif countryNum == 2 &&
~isempty(app.Country2Data)
            params = [app.K2Slider.Value, app.r2Slider.Value,
app.t02Slider.Value];
            plotModelResults(app, 2, app.Country2Data, params);
        end
    end

    % Reset slider functions
    function ResetSlidersButtonPushed(app, event)
        if ~isempty(app.Country1Data)
            updateSliderRanges(app, 1, app.Country1Data);
            updateModelPlot(app, 1);
            updateProgress(app, 100, 'Country 1 sliders reset');
            app.StatusLabel.FontColor = app.successColor;
        end
    end

    function ResetSliders2ButtonPushed(app, event)
        if ~isempty(app.Country2Data)
            updateSliderRanges(app, 2, app.Country2Data);
            updateModelPlot(app, 2);
            updateProgress(app, 100, 'Country 2 sliders reset');
            app.StatusLabel.FontColor = app.successColor;
        end
    end
end

```

```

% Export functions
function Export1ButtonPushed(app, event)
if isempty(app.Country1Model)
    uialert(app.UIFigure, 'Please fit the model first', 'No
Model Data');
    return;
end

[file, path] = uiputfile({'...
'*.*csv', 'CSV Files (*.csv)'; ...
'*.*xlsx', 'Excel Files (*.xlsx)'; ...
'*.*mat', 'MAT Files (*.mat)'}, ...
'Export Country 1 Results');

if file
    try
        data = app.Country1Model.data;
        params = app.Country1Model.params;
        logistic_model = @(p, t) p(1) ./ (1 + exp(-p(2) * (t -
p(3))));
        predicted = logistic_model(params,
data.TimeDays);

        exportData = table(data.Date, data.Cases, predicted,
...
        data.Cases - predicted, ...
        'VariableNames', {'Date', 'Actual', 'Predicted',
'Residual'});

        [~, ~, ext] = fileparts(file);
        fullPath = fullfile(path, file);

        switch lower(ext)
            case '.csv'
                writetable(exportData, fullPath);
            case '.xlsx'
                writetable(exportData, fullPath);
            case '.mat'
                results = struct('data', exportData, 'parameters',
params);
                save(fullPath, 'results');
            end

        updateProgress(app, 100, 'Country 1 results
exported successfully');
        app.StatusLabel.FontColor = app.successColor;
        catch ME
            uialert(app.UIFigure, ME.message, 'Export Error');
        end
    end
end

function Export2ButtonPushed(app, event)
if isempty(app.Country2Model)
    uialert(app.UIFigure, 'Please fit the model first', 'No
Model Data');
    return;
end

[file, path] = uiputfile({'...
'*.*csv', 'CSV Files (*.csv)'; ...
'*.*xlsx', 'Excel Files (*.xlsx)'; ...
'*.*mat', 'MAT Files (*.mat)'}, ...
'Export Country 2 Results');

if file
    try
        data = app.Country2Model.data;
        params = app.Country2Model.params;
        logistic_model = @(p, t) p(1) ./ (1 + exp(-p(2) * (t -
p(3))));
        predicted = logistic_model(params,
data.TimeDays);

```

```

        exportData = table(data.Date, data.Cases, predicted,
...
        data.Cases - predicted, ...
        'VariableNames', {'Date', 'Actual', 'Predicted',
'Residual'});

        [~, ~, ext] = fileparts(file);
        fullPath = fullfile(path, file);

        switch lower(ext)
            case '.csv'
                writetable(exportData, fullPath);
            case '.xlsx'
                writetable(exportData, fullPath);
            case '.mat'
                results = struct('data', exportData, 'parameters',
params);
                save(fullPath, 'results');
            end

        updateProgress(app, 100, 'Country 2 results
exported successfully');
        app.StatusLabel.FontColor = app.successColor;
        catch ME
            uialert(app.UIFigure, ME.message, 'Export Error');
        end
    end
end

function updateComparisonPlot(app)
if ~isempty(app.Country1Model) &&
~isempty(app.Country2Model)
    cla(app.ComparisonAxes);
    hold(app.ComparisonAxes, 'on');
    if app.isDarkMode
        plot(app.ComparisonAxes,
app.Country1Model.data.Date, ...
app.Country1Model.data.Cases, 'o-', 'Color', [0 1
1], ... % cyan
        'DisplayName', app.Country1DropDown.Value,
'LineWidth', 2);
        plot(app.ComparisonAxes,
app.Country2Model.data.Date, ...
app.Country2Model.data.Cases, 's-', 'Color', [1 0
1], ... % magenta
        'DisplayName', app.Country2DropDown.Value,
'LineWidth', 2);
        title(app.ComparisonAxes, 'Country Comparison',
'Color', [1 1 1]);
        app.ComparisonAxes.XColor = [1 1 1];
        app.ComparisonAxes.YColor = [1 1 1];
        lgd = legend(app.ComparisonAxes, 'Location',
'northwest');
        lgd.TextColor = [1 1 1]; lgd.Color = [0.15 0.18
0.22];
    else
        plot(app.ComparisonAxes,
app.Country1Model.data.Date, ...
app.Country1Model.data.Cases, 'o-', 'Color', [0 0
1], ... % blue
        'DisplayName', app.Country1DropDown.Value,
'LineWidth', 2);
        plot(app.ComparisonAxes,
app.Country2Model.data.Date, ...
app.Country2Model.data.Cases, 's-', 'Color', [1 0
0], ... % red
        'DisplayName', app.Country2DropDown.Value,
'LineWidth', 2);
        title(app.ComparisonAxes, 'Country Comparison',
'Color', [0 0 0]);
        app.ComparisonAxes.XColor = [0.1 0.1 0.1];
        app.ComparisonAxes.YColor = [0.1 0.1 0.1];
        lgd = legend(app.ComparisonAxes, 'Location',
'northwest');
        lgd.TextColor = [0.1 0.1 0.1]; lgd.Color = [1 1 1];
    end
end

```



```

end
xlabel(app.ComparisonAxes, 'Date');
ylabel(app.ComparisonAxes, 'Cumulative Cases');
grid(app.ComparisonAxes, 'on');
hold(app.ComparisonAxes, 'off');
updateComparisonMetrics(app);
end
end

function updateComparisonMetrics(app)
if ~isempty(app.Country1Model) &&
~isempty(app.Country2Model)
params1 = app.Country1Model.params;
params2 = app.Country2Model.params;

metricsText = sprintf([...
'%s:\n' ...
'K: %.0f, r: %.3f, to: %.1f\n' ...
'%s:\n' ...
'K: %.0f, r: %.3f, to: %.1f\n' ...
'Comparison:\n' ...
'K ratio: %.2f\n' ...
'r ratio: %.2f'], ...
app.Country1DropDown.Value, params1(1),
params1(2), params1(3), ...
app.Country2DropDown.Value, params2(1),
params2(2), params2(3), ...
params1(1)/params2(1), params1(2)/params2(2));

app.ComparisonMetricsLabel.Text = metricsText;
end
end

function ExportComparisonButtonPushed(app, event)
if isempty(app.Country1Model) ||
isempty(app.Country2Model)
uialert(app.UIFigure, 'Please fit models for both
countries first', 'No Comparison Data');
return;
end

[file, path] = uiputfile({'*.csv', 'CSV Files'; '*.xlsx',
'Excel Files'}, ...
'Export Comparison Results');

if file
try
% Create comparison table
data1 = app.Country1Model.data;
data2 = app.Country2Model.data;
params1 = app.Country1Model.params;
params2 = app.Country2Model.params;

% Parameters comparison
paramTable = table(...
{app.Country1DropDown.Value;
app.Country2DropDown.Value}, ...
[params1(1); params2(1)], [params1(2);
params2(2)], [params1(3); params2(3)], ...
'VariableNames', {'Country',
'K_CarryingCapacity', 'r_GrowthRate', 't0_InflexionPoint'});

writetable(paramTable, fullfile(path, file));

updateProgress(app, 100, ' Comparison results
exported successfully');
app.StatusLabel.FontColor = app.successColor;
catch ME
uialert(app.UIFigure, ME.message, 'Export Error');
end
end
end

% --- [5] ADVANCED ANALYTICS DISPATCHER &
UTILITY FUNCTIONS (private)

```

```

function runAdvAnalysis(app)
t = app.AdvAnalysisDropDown.Value;
switch t
case 'Country Dashboard'
showCountryDashboard(app);
case 'World Dashboard'
showWorldDashboard(app);
case 'Date Statistics'
showDateStats(app);
case 'Compare Countries'
showCountryComparison(app);
case 'Saturation Dates'
showSaturationDates(app);
end
end

function showCountryDashboard(app)
c = app.AdvCountry1DropDown.Value;
if isempty(c) || strcmp(c, 'Select Country')
if ~isempty(app.AdvResultLabel)
app.AdvResultLabel.Text = 'Please select a
country.';
end
return;
end
data = extractCountryData(app, c);
pars = fitLogistic(app, data);
plotAdvFourPanel(app, data, pars, sprintf('%s
Dashboard', c));
end

function showWorldDashboard(app)
data = aggregateWorldData(app);
pars = fitLogistic(app, data);
plotAdvFourPanel(app, data, pars, 'World Dashboard');
end

function showDateStats(app)
dtx = app.AdvDateField.Value;
try
target = datetime(dtx, 'InputFormat', 'yyyy-MM-dd');
catch
if ~isempty(app.AdvResultLabel)
app.AdvResultLabel.Text = 'Use YYYY-MM-DD
format';
app.AdvResultLabel.FontSize = 20;
if app.isDarkMode
app.AdvResultLabel.FontColor = [1 1 1];
else
app.AdvResultLabel.FontColor = [0.1 0.1 0.1];
end
end
return;
end

% Determine columns
countryCol = findColumn(app, app.DataTable,
{'location', 'country', 'Country_Region', 'Entity', 'nation',
'region'});
dateCol = findColumn(app, app.DataTable, {'date',
'Date', 'dateRep', 'Day'});
casesCol = findColumn(app, app.DataTable,
{'cases', 'Cases', 'total_cases', 'Cumulative_cases'});
deathsCol = findColumn(app,
app.DataTable, {'deaths', 'Deaths'});
tbl = app.DataTable;
selectedCountry = app.AdvCountry1DropDown.Value;
dd = tbl.(dateCol); if ~isdatetime(dd), dd =
datetime(dd, 'InputFormat', 'auto'); end
[~, irow] = min(abs(dd-target));
if strcmp(selectedCountry, 'World') ||
isempty(selectedCountry) || any(strcmpi(selectedCountry,
{'Select Country', 'Select Country 1'}))
% World statistics for the date
mask = dd == dd(irow);
totalCases = sum(tbl.(casesCol)(mask), 'omitnan');
totalDeaths = 0;
if ~isempty(deathsCol),
totalDeaths = sum(tbl.(deathsCol)(mask), 'omitnan'); end

```

```

        if ~isempty(app.AdvResultLabel)
            app.AdvResultLabel.Text = sprintf('Date:
%s\nWorldwide Infected: %d\nWorldwide Deaths: %d', ...
            datestr(dd(irow)), totalCases, totalDeaths);
            app.AdvResultLabel.FontSize = 20;
            if app.isDarkMode
                app.AdvResultLabel.FontColor = [1 1 1];
            else
                app.AdvResultLabel.FontColor = [0.1 0.1 0.1];
            end
        end
    else
        mask = strcmp(tbl.(countryCol), selectedCountry) &
dd == dd(irow);
        totalCases = sum(tbl.(casesCol)(mask), 'omitnan');
        totalDeaths = 0;
        if ~isempty(deathsCol),
totalDeaths=sum(tbl.(deathsCol)(mask), 'omitnan'); end
        if ~isempty(app.AdvResultLabel)
            app.AdvResultLabel.Text = sprintf('Date:
%s\nTotal Infected: %d\nTotal Deaths: %d', ...
            datestr(dd(irow)), totalCases, totalDeaths);
            app.AdvResultLabel.FontSize = 20;
            if app.isDarkMode
                app.AdvResultLabel.FontColor = [1 1 1];
            else
                app.AdvResultLabel.FontColor = [0.1 0.1 0.1];
            end
        end
    end
end
function showCountryComparison(app)
    c1=app.AdvCountry1Dropdown.Value;
c2=app.AdvCountry2Dropdown.Value;
    if any(strcmp('Select Country', ''), {c1,c2})) ||
strcmp(c1,c2)
        if ~isempty(app.AdvResultLabel)
            app.AdvResultLabel.Text = 'Select two different
countries.';
            app.AdvResultLabel.FontSize = 20;
            if app.isDarkMode
                app.AdvResultLabel.FontColor = [1 1 1];
            else
                app.AdvResultLabel.FontColor = [0.1 0.1 0.1];
            end
        end
    end
    return;
end
d1=extractCountryData(app,c1);
d2=extractCountryData(app,c2);
pars1=fitLogistic(app,d1); pars2=fitLogistic(app,d2);
axesList={app.AdvAxes1,app.AdvAxes2};
cla(app.AdvAxes1); cla(app.AdvAxes2);
plot(axesList{1}, d1.Date, d1.Cases, 'b-
o','DisplayName',c1);
hold(axesList{1},'on');
plot(axesList{1},d1.Date,pars1.fitted,'r-');
hold(axesList{1},'off');
plot(axesList{2}, d2.Date, d2.Cases, 'g-
o','DisplayName',c2);
hold(axesList{2},'on');
plot(axesList{2},d2.Date,pars2.fitted,'m-');
hold(axesList{2},'off');
title(axesList{1},[c1 ' Model']);
title(axesList{2},[c2 ' Model']);
if ~isempty(app.AdvResultLabel)
    app.AdvResultLabel.Text = sprintf('K=%f
r=%f t0=%f. If n[%s]nK=%f r=%f t0=%f. ...
c1,pars1.K,pars1.r,pars1.t0,c2,pars2.K,pars2.r,pars2.t0);
    app.AdvResultLabel.FontSize = 20;
    if app.isDarkMode
        app.AdvResultLabel.FontColor = [1 1 1];
    else
        app.AdvResultLabel.FontColor = [0.1 0.1 0.1];
    end
end

```

```

    end
end
function showSaturationDates(app)
    c=app.AdvCountry1Dropdown.Value;
    if isempty(c) || strcmp(c,'Select Country')
        if ~isempty(app.AdvResultLabel)
            app.AdvResultLabel.Text='Select a country.';
            app.AdvResultLabel.FontSize = 20;
            if app.isDarkMode
                app.AdvResultLabel.FontColor = [1 1 1];
            else
                app.AdvResultLabel.FontColor = [0.1 0.1 0.1];
            end
        end
    end
    return;
end
data=extractCountryData(app,c);
pars=fitLogistic(app,data);
K=pars.K;
logistic=@(p,t) p(1)./(1+exp(-p(2)*(t-p(3))));
t_future = linspace(0, max(data.TimeDays)*2, 2000);
curve = logistic([pars.K pars.r pars.t0], t_future);
sat95 = 0.95*pars.K;
idx = find(curve>=sat95,1);
date_sat = data.Date(1)+days(t_future(idx));
if ~isempty(app.AdvResultLabel)
    app.AdvResultLabel.Text = sprintf('%s\nSaturation
(95%%)=%f\nPredicted Date: %s',c,sat95,datestr(date_sat));
    app.AdvResultLabel.FontSize = 20;
    if app.isDarkMode
        app.AdvResultLabel.FontColor = [1 1 1];
    else
        app.AdvResultLabel.FontColor = [0.1 0.1 0.1];
    end
end
end
function showAdvMetrics(app)
    c=app.AdvCountry1Dropdown.Value;
    if isempty(c) || strcmp(c,'Select Country')
        if ~isempty(app.AdvResultLabel)
            app.AdvResultLabel.Text = 'Select a country.';
            app.AdvResultLabel.FontSize = 20;
            if app.isDarkMode
                app.AdvResultLabel.FontColor = [1 1 1];
            else
                app.AdvResultLabel.FontColor = [0.1 0.1 0.1];
            end
        end
    end
    return;
end
data=extractCountryData(app,c);
pars=fitLogistic(app,data);
if ~isempty(app.AdvResultLabel)
    app.AdvResultLabel.Text =
sprintf('K=%f\nr=%f\nt0=%f\nRMSE=%f',pars.K,pars.r,
pars.t0,pars.rmse);
    app.AdvResultLabel.FontSize = 20;
    if app.isDarkMode
        app.AdvResultLabel.FontColor = [1 1 1];
    else
        app.AdvResultLabel.FontColor = [0.1 0.1 0.1];
    end
end
end
function plotAdvFourPanel(app,data,pars,mainTitle)
    logistic=@(p,t) p(1)./(1+exp(-p(2)*(t-p(3))));
    t_data = data.TimeDays; dates = data.Date;
    N_data = data.Cases; N_pred = pars.fitted;
    % Actual vs model
    cla(app.AdvAxes1); plot(app.AdvAxes1, dates,
N_data,'bo-',dates,N_pred,'r-');
    title(app.AdvAxes1,'Actual vs Model');
    app.AdvAxes1.Title.Color = [1 0 0];
    % Residuals

```

```

        cla(app.AdvAxes2); plot(app.AdvAxes2,dates,N_data-
N_pred,'ko-'); ylabel(app.AdvAxes2,0,'-r');
        title(app.AdvAxes2, 'Residuals');
        app.AdvAxes2.Title.Color = [1 0 0];
        % Growth rate
        gr = pars.r * N_pred .* (1 - N_pred/pars.K);
        cla(app.AdvAxes3); plot(app.AdvAxes3,dates,gr,'g-');
        title(app.AdvAxes3, 'Growth Rate');
        app.AdvAxes3.Title.Color = [1 0 0];
        % Sensitivity
        t_test =
linspace(min(t_data),max(t_data),numel(t_data));
        N_sens=logistic([pars.K*0.8 pars.r pars.t0],t_test);
        cla(app.AdvAxes4);
        plot(app.AdvAxes4, t_test, N_sens,'b--');
        hold(app.AdvAxes4,'on');
        plot(app.AdvAxes4, t_test, logistic([pars.K pars.r
pars.t0],t_test),'r-');
        plot(app.AdvAxes4, t_test, logistic([pars.K*1.2 pars.r
pars.t0],t_test),'g--');
        hold(app.AdvAxes4,'off'); title(app.AdvAxes4,'K
parameter Sensitivity');
        app.AdvAxes4.Title.Color = [1 0 0];
        % Set metrics:
        if ~isempty(app.AdvResultLabel)
            app.AdvResultLabel.Text = sprintf('Carrying
Capacity=%0.0f\nGrowth
Rate=%0.4f\nRMSE=%0.2f\nR2=%0.3f',pars.K,pars.r,pars.rmse,pars.r2);
            app.AdvResultLabel.FontSize = 20;
            if app.isDarkMode
                app.AdvResultLabel.FontColor = [1 1 1];
            else
                app.AdvResultLabel.FontColor = [0.1 0.1 0.1];
            end
        end
    end
end
function pars=fitLogistic(app,data)
    t=data.TimeDays; N=data.Cases;
    mask = N>0;
    tfit=t(mask); Nfit=N(mask);
    logistic=@(p,t) p(1)/(1+exp(-p(2)*(t-p(3))));
    init=[max(N)*1.5, 0.1, tfit(round(end/2))];
    lb = [max(N), 0.01, 0]; ub=[max(N)*10, 1,
max(tfit)*1.5];

options=optimoptions('lsqcurvefit','Display','off','MaxIter',1000);
[p,~,~,exitflag]=lsqcurvefit(logistic, init, tfit, Nfit, lb, ub,
options);
    if exitflag<=0, objfun=@(p) sum((logistic(p, tfit)-
Nfit).^2); p=fminsearch(objfun,init); end
    % Predict for all days
    fitted=logistic(p,t);
    resid=N-fitted; rmse=sqrt(mean(resid.^2)); r2=1-
sum(resid.^2)/sum((N-mean(N)).^2);

pars=struct('K',p(1),'r',p(2),'t0',p(3),'rmse',rmse,'r2',r2,'fitted',fitted);
end
function data=aggregateWorldData(app)
    colCountry=findColumn(app,
app.DataTable,{'location','country','Country_Region','Entity','nation',
'region'});
    colDate=findColumn(app,
app.DataTable,{'date','Date','dateRep','time','Day','Date_reported'
});
    colCases=findColumn(app,
app.DataTable,{'total_cases','Cases','cases','Cumulative_cases','cumulative_cases',
'Total_cases'});
    t=app.DataTable;
    dates=t.colDate;
    if ~isdatetime(dates),
dates=datetime(dates,'InputFormat','auto'); end
    cases=t.colCases; if iscell(cases),
cases=str2double(cases); end

```

```

G=groupsummary(table(dates,cases),'dates','sum','cases');
timeDays=days(G.dates-G.dates(1));

data=table(G.dates,G.sum_cases,timeDays,'VariableNames',{'Date','Cases','TimeDays'});
end

% --- FIX: This function now handles the
ValueChangingFcn event for live filtering ---
function filterCountryDropdown(app, src, event)
    searchText = lower(event.Value); % Use event data for
live filtering

    if isempty(searchText)
        filtered = app.CountryList;
    else
        filtered =
app.CountryList(contains(lower(app.CountryList), searchText));
    end

    if isequal(src, app.Country1DropDown)
        src.Items = {'Select Country 1'}, filtered;
    elseif isequal(src, app.Country2DropDown)
        src.Items = {'Select Country 2'}, filtered;
    elseif isequal(src, app.AdvCountry1DropDown)
        src.Items = {'Select Country'}, filtered;
    elseif isequal(src, app.AdvCountry2DropDown)
        src.Items = {'Select Country (Only for comparison)'},
filtered;
    end
end

function showAdvAbout(app)
    msg = [ ...
        '===== \n', ...
        ' Advanced Analytics Guide \n', ...
        '===== \n', ...
        '\n', ...
        'This section provides powerful tools to explore
COVID-19 data in depth.\n', ...
        '\n', ...
        'Analysis Options:\n', ...
        '----- \n', ...
        '- Country Dashboard:\n', ...
        ' A four-part visual summary for a single country.\n',
...
        '\n', ...
        '- World Dashboard:\n', ...
        ' Aggregates all countries for a global overview.\n', ...
        '\n', ...
        '- Date Statistics:\n', ...
        ' Select a date and country (or World) to see total
cases and deaths up to that day.\n', ...
        '\n', ...
        '- Compare Countries:\n', ...
        ' Side-by-side plots and model fits for two
countries.\n', ...
        '\n', ...
        '- Saturation Dates:\n', ...
        ' Predicts when a country will reach 95% of its total
predicted cases (K).\n', ...
        '\n', ...
        '- Metrics Only:\n', ...
        ' Shows model parameters (K, r, t0) and error stats
(RMSE, R2) for a country.\n', ...
        '\n', ...
        'Understanding the Graphs:\n', ...
        '----- \n', ...
        '- Actual vs. Model:\n', ...
        ' Blue dots: actual data. Red line: fitted logistic
model.\n', ...
        '\n', ...
        '- Residuals:\n', ...
        ' Difference between actual and model for each

```

```

date.\n', ...
    ' Points above zero: model under-predicted.\n', ...
    ' Points below zero: model over-predicted.\n', ...
    '\n', ...
    '- Growth Rate:\n', ...
    ' Estimated new cases per day. Peak = inflection point
(t0).\n', ...
    '\n', ...
    '- K Parameter Sensitivity:\n', ...
    ' Shows how predictions change if K is 20% lower or
higher.\n', ...
    '\n', ...
    'Key Metrics Explained:\n', ...
    '-----\n', ...
    '- K (Carrying Capacity):\n', ...
    ' Predicted total cases at the end of the pandemic.\n',
...
    '- r (Growth Rate):\n', ...
    ' How fast the pandemic spreads.\n', ...
    '- t0 (Inflection Point):\n', ...
    ' Day when new cases peaked.\n', ...
    '- RMSE (Root Mean Square Error):\n', ...
    ' Average model error. Lower is better.\n', ...
    '- R2 (R-squared):\n', ...
    ' 0 to 1. Higher means model fits data better.\n', ...
    '\n', ...
    'Tips:\n', ...
    '-----\n', ...
    '- Use dropdowns to select countries and analysis
types.\n', ...
    '- You can type in dropdowns to filter country
names.\n', ...
    '\n', ...
    'For more help, see documentation or contact the
developer.' ...
    ];
    uialert(app.UIFigure, msg, 'About Advanced Analytics',
'Icon', 'info');
end

% --- PDF Report Generation for Country 1 ---
function GeneratePDFReport1ButtonPushed(app, event)
    generateCountryPDFReport(app, 1);
end
% --- PDF Report Generation for Country 2 ---
function GeneratePDFReport2ButtonPushed(app, event)
    generateCountryPDFReport(app, 2);
end
function generateCountryPDFReport(app, countryNum)
    if countryNum == 1
        model = app.Country1Model;
        country = app.Country1DropDown.Value;
        data = app.Country1Data;
        metricsText = app.Metrics1Label.Text;
        params = app.Country1Model.params;
    else
        model = app.Country2Model;
        country = app.Country2DropDown.Value;
        data = app.Country2Data;
        metricsText = app.Metrics2Label.Text;
        params = app.Country2Model.params;
    end
    if isempty(model)
        uialert(app.UIFigure, 'Please fit the model first', 'No
Model Data');
        return;
    end
    [file, path] = uiputfile({'*.pdf','PDF Files'}, 'Save PDF
Report As', [country ' _COVID19_Report.pdf']);
    if isequal(file,0)
        return;
    end
    fullPath = fullfile(path, file);

    % Create a temporary figure with 2x2 subplots and re-

```

```

plot the data
tempFig = figure('Visible','off','Position',[100 100 1200
900]);
% 1. Raw Data
subplot(2,2,1);
plot(data.Date, data.Cases, 'o-', 'Color', [0 0.447 0.741],
'LineWidth', 2, 'MarkerSize', 4);
title([country ' - Raw Data']);
xlabel('Date'); ylabel('Cumulative Cases'); grid on;
% 2. Model Fit
subplot(2,2,2);
t_data = data.TimeDays;
N_data = data.Cases;
logistic_model = @(params, t) params(1) / (1 + exp(-
params(2) * (t - params(3))));
N_predicted = logistic_model(params, t_data);
plot(data.Date, N_data, 'o', 'Color', [0 0.447 0.741],
'MarkerSize', 4, 'DisplayName', 'Actual Data'); hold on;
plot(data.Date, N_predicted, '-', 'Color', [0.85 0.33 0.1],
'LineWidth', 2, 'DisplayName', 'Model Fit');
title('Logistic Model Fit'); xlabel('Date');
ylabel('Cumulative Cases'); grid on; legend('show'); hold off;
% 3. Residuals
subplot(2,2,3);
residuals = N_data - N_predicted;
plot(data.Date, residuals, 'o-', 'Color', [0.494 0.184
0.556], 'MarkerSize', 3);
yline(0, '--', 'Color', [0.5 0.5 0.5]);
title('Residuals Analysis'); xlabel('Date');
ylabel('Residuals'); grid on;
% 4. Future Predictions
subplot(2,2,4);
t_future = linspace(0, max(t_data) * 1.5, 500);
N_future = logistic_model(params, t_future);
date_future = data.Date(1) + days(t_future);
plot(data.Date, N_data, 'o', 'Color', [0 0.447 0.741],
'MarkerSize', 4, 'DisplayName', 'Actual Data'); hold on;
plot(date_future, N_future, '-', 'Color', [0.466 0.674
0.188], 'LineWidth', 2, 'DisplayName', 'Prediction');
title('Future Predictions'); xlabel('Date');
ylabel('Cumulative Cases'); grid on; legend('show'); hold off;
sgtitle(['COVID-19 Logistic Model Report: ' country]);

% Export the figure as a single PDF
exportgraphics(tempFig, fullPath, 'ContentType',
'vector');
close(tempFig);

% Export metrics as a text file
metricsFile = strep(fullfile, '.pdf', '_metrics.txt');
fid = fopen(metricsFile, 'w');
fprintf(fid, '%s', metricsText);
fclose(fid);

uialert(app.UIFigure, ['PDF with all plots generated: '
fullPath char(10) 'Metrics saved as: ' metricsFile], 'Report
Complete');
end

function addEventMarker(app, countryNum)
    if countryNum == 1
        dateStr = app.EventMarkerDate1.Value;
        label = app.EventMarkerLabel1.Value;
        if strcmp(dateStr, 'Select Date') || isempty(label)
            return;
        end
        dateVal = datetime(dateStr);
        if isempty(app.EventMarkers1)
            app.EventMarkers1 = table(dateVal, {label},
'VariableNames', {'Date','Label'});
        else
            app.EventMarkers1 = [app.EventMarkers1;
{dateVal, label}];
        end
        app.EventMarkerLabel1.Value = '';
    end
end

```

```

        updateMarkerList(app, 1);
        plotCountryData(app, 1, app.Country1Data);
    else
        dateStr = app.EventMarkerDate2.Value;
        label = app.EventMarkerLabel2.Value;
        if strcmp(dateStr, 'Select Date') || isempty(label)
            return;
        end
        dateVal = datetime(dateStr);
        if isempty(app.EventMarkers2)
            app.EventMarkers2 = table(dateVal, {label},
'VariableNames', {'Date','Label'});
        else
            app.EventMarkers2 = [app.EventMarkers2;
{dateVal, label}];
        end
        app.EventMarkerLabel2.Value = "";
        updateMarkerList(app, 2);
        plotCountryData(app, 2, app.Country2Data);
    end
end

function removeEventMarker(app, countryNum)
    if countryNum == 1
        if isempty(app.EventMarkers1)
            return;
        end
        % Remove the last marker (or implement a UI for
selection if needed)
        app.EventMarkers1(end,:) = [];
        plotCountryData(app, 1, app.Country1Data);
    else
        if isempty(app.EventMarkers2)
            return;
        end
        app.EventMarkers2(end,:) = [];
        plotCountryData(app, 2, app.Country2Data);
    end
end

function updateMarkerList(app, countryNum)
    % No-op: listbox removed
end

% Update event marker date dropdowns when country data
is loaded
function updateEventMarkerDates(app, countryNum, data)
    dateStrs = cellstr(string(data.Date));
    if countryNum == 1
        app.EventMarkerDate1.Items = {'Select Date'},
dateStrs{:};
    else
        app.EventMarkerDate2.Items = {'Select Date'},
dateStrs{:};
    end
end

% Component initialization
methods (Access = private)

% Create UIFigure and components
function createComponents(app)
    % Create UIFigure and hide until all components are
created
    app.UIFigure = uifigure('Visible', 'off');
    app.UIFigure.Position = [100 50 1600 1000];
    app.UIFigure.Name = 'COVID-19 Logistic Model
Analysis Suite - Fixed';
    app.UIFigure.Resize = 'on';

    % Create UploadButton
    app.UploadButton = uibutton(app.UIFigure, 'push');
    app.UploadButton.ButtonPushedFcn =
createCallbackFcn(app, @UploadButtonPushed, true);

```

```

    app.UploadButton.Position = [30 950 140 35];
    app.UploadButton.Text = 'Upload Data';
    app.UploadButton.FontSize = 12;

    % Create Country Dropdowns
    app.Country1DropDownLabel = uilabel(app.UIFigure);
    app.Country1DropDownLabel.HorizontalAlignment =
'right';
    app.Country1DropDownLabel.Position = [190 955 90
25]; % moved left
    app.Country1DropDownLabel.Text = 'Country 1:';
    app.Country1DropDownLabel.FontSize = 12;

    app.Country1DropDown = uidropdown(app.UIFigure);
    app.Country1DropDown.Items = {'Select Country 1'};
    app.Country1DropDown.Position = [290 952 200 30]; %
moved left
    app.Country1DropDown.FontSize = 15;
    % --- FIX: Assign callbacks to the correct properties ---
    app.Country1DropDown.Editable = true;
    app.Country1DropDown.ValueChangedFcn =
createCallbackFcn(app, @Country1DropDownValueChanged,
true);

    app.Country2DropDownLabel = uilabel(app.UIFigure);
    app.Country2DropDownLabel.HorizontalAlignment =
'right';
    app.Country2DropDownLabel.Position = [510 955 90
25]; % moved left
    app.Country2DropDownLabel.Text = 'Country 2:';
    app.Country2DropDownLabel.FontSize = 12;

    app.Country2DropDown = uidropdown(app.UIFigure);
    app.Country2DropDown.Items = {'Select Country 2'};
    app.Country2DropDown.Position = [610 952 200 30]; %
moved left
    app.Country2DropDown.FontSize = 15;
    % --- FIX: Assign callbacks to the correct properties ---
    app.Country2DropDown.Editable = true;
    app.Country2DropDown.ValueChangedFcn =
createCallbackFcn(app, @Country2DropDownValueChanged,
true);

    % Create SwapButton
    app.SwapButton = uibutton(app.UIFigure, 'push');
    app.SwapButton.ButtonPushedFcn =
createCallbackFcn(app, @SwapButtonPushed, true);
    app.SwapButton.Position = [830 950 80 35]; % moved
left
    app.SwapButton.Text = 'Swap';
    app.SwapButton.FontSize = 11;

    % Add Dark/Light Mode button to header bar
    app.AdvThemeButton = uibutton(app.UIFigure, 'push',
'Position', [1450 950 90 35], ...
'Text', 'ButtonPushedFcn',
@(src,evt)toggleTheme(app));
    % Add About button to header bar
    app.AdvAboutButton = uibutton(app.UIFigure, 'push',
'Position', [1550 950 90 35], ...
'Text', 'About', 'ButtonPushedFcn',
@(src,evt)showAdvAbout(app));

    % Create Progress Gauge
    app.ProgressGauge = uigauge(app.UIFigure, 'linear');
    app.ProgressGauge.Position = [930 955 300 25]; %
moved left
    app.ProgressGauge.Limits = [0 100];

    % Create TabGroup
    app.TabGroup = uitabgroup(app.UIFigure);
    app.TabGroup.Position = [20 80 1560 850];

    % Create Country1Tab
    app.Country1Tab = uitab(app.TabGroup);

```

```

app.Country1Tab.Title = 'us Country 1 Analysis';

% Create Axes1_1 (Raw Data)
app.Axes1_1 = uiaxes(app.Country1Tab);
app.Axes1_1.Position = [30 450 400 350];
title(app.Axes1_1, 'Raw Data');

% Create Axes1_2 (Model Fit)
app.Axes1_2 = uiaxes(app.Country1Tab);
app.Axes1_2.Position = [450 450 400 350];
title(app.Axes1_2, 'Logistic Model Fit');

% Create Axes1_3 (Residuals)
app.Axes1_3 = uiaxes(app.Country1Tab);
app.Axes1_3.Position = [30 80 400 350];
title(app.Axes1_3, 'Residuals Analysis');

% Create Axes1_4 (Predictions)
app.Axes1_4 = uiaxes(app.Country1Tab);
app.Axes1_4.Position = [450 80 400 350];
title(app.Axes1_4, 'Future Predictions');

% Slider controls for Country 1
% K1 Slider
app.K1SliderLabel = uilabel(app.Country1Tab);
app.K1SliderLabel.Position = [1280 720 200 22];
app.K1SliderLabel.Text = 'K (Carrying Capacity)';
app.K1SliderLabel.FontWeight = 'bold';
app.K1SliderLabel.FontSize = 15;

app.K1Slider = uislider(app.Country1Tab);
app.K1Slider.Position = [900 720 300 3];
app.K1Slider.Limits = [0 1000000];
app.K1Slider.ValueChangedFcn =
createCallbackFcn(app, @K1SliderValueChanged, true);
if app.isDarkMode
    app.K1Slider.FontColor = [1 1 1];
else
    app.K1Slider.FontColor = [0.1 0.1 0.1];
end

app.K1ValueLabel = uilabel(app.Country1Tab);
app.K1ValueLabel.Position = [1220 720 100 22];
app.K1ValueLabel.Text = '0';
app.K1ValueLabel.FontWeight = 'bold';

% r1 Slider
app.r1SliderLabel = uilabel(app.Country1Tab);
app.r1SliderLabel.Position = [1280 650 200 22];
app.r1SliderLabel.Text = 'r (Growth Rate)';
app.r1SliderLabel.FontWeight = 'bold';
app.r1SliderLabel.FontSize = 15;

app.r1Slider = uislider(app.Country1Tab);
app.r1Slider.Position = [900 650 300 3];
app.r1Slider.Limits = [0.001 2.0];
app.r1Slider.ValueChangedFcn =
createCallbackFcn(app, @r1SliderValueChanged, true);
if app.isDarkMode
    app.r1Slider.FontColor = [1 1 1];
else
    app.r1Slider.FontColor = [0.1 0.1 0.1];
end

app.r1ValueLabel = uilabel(app.Country1Tab);
app.r1ValueLabel.Position = [1220 650 100 22];
app.r1ValueLabel.Text = '0.000';
app.r1ValueLabel.FontWeight = 'bold';

% t01 Slider
app.t01SliderLabel = uilabel(app.Country1Tab);
app.t01SliderLabel.Position = [1280 580 200 22];
app.t01SliderLabel.Text = 't0 (Inflection Point)';
app.t01SliderLabel.FontWeight = 'bold';
app.t01SliderLabel.FontSize = 15;

app.t01Slider = uislider(app.Country1Tab);
app.t01Slider.Position = [900 580 300 3];
app.t01Slider.Limits = [1 365];
app.t01Slider.ValueChangedFcn =
createCallbackFcn(app, @t01SliderValueChanged, true);
if app.isDarkMode
    app.t01Slider.FontColor = [1 1 1];
else
    app.t01Slider.FontColor = [0.1 0.1 0.1];
end

app.t01ValueLabel = uilabel(app.Country1Tab);
app.t01ValueLabel.Position = [1220 580 100 22];
app.t01ValueLabel.Text = '1';
app.t01ValueLabel.FontWeight = 'bold';

% Control buttons for Country 1
app.FitModelButton = uibutton(app.Country1Tab,
'push');
app.FitModelButton.ButtonPushedFcn =
createCallbackFcn(app, @FitModelButtonPushed, true);
app.FitModelButton.Position = [900 500 140 35];
app.FitModelButton.Text = 'Auto Fit Model';
app.FitModelButton.FontWeight = 'bold';

app.ResetSlidersButton = uibutton(app.Country1Tab,
'push');
app.ResetSlidersButton.ButtonPushedFcn =
createCallbackFcn(app, @ResetSlidersButtonPushed, true);
app.ResetSlidersButton.Position = [1060 500 140 35];
app.ResetSlidersButton.Text = 'Reset Sliders';

% Export button for Country 1
app.Export1Button = uibutton(app.Country1Tab, 'push');
app.Export1Button.ButtonPushedFcn =
createCallbackFcn(app, @Export1ButtonPushed, true);
app.Export1Button.Position = [900 450 200 35];
app.Export1Button.Text = 'Export Results';

% Generate PDF Report button for Country 1
app.Report1Button = uibutton(app.Country1Tab, 'push');
app.Report1Button.ButtonPushedFcn =
createCallbackFcn(app, @GeneratePDFReport1ButtonPushed,
true);
app.Report1Button.Position = [900 400 200 35];
app.Report1Button.Text = 'Generate PDF Report';

% Metrics display for Country 1 (shifted down to avoid
overlap)
app.Metrics1Label = uilabel(app.Country1Tab);
app.Metrics1Label.Position = [900 20 500 280];
app.Metrics1Label.Text = 'Model metrics will appear
here...';
app.Metrics1Label.VerticalAlignment = 'top';
app.Metrics1Label.FontName = 'Courier New';
app.Metrics1Label.FontSize = 10;

% --- Country 1 Event Marker Controls ---
app.EventMarkerDate1 = uidropdown(app.Country1Tab,
'Position', [900 370 120 25], 'Items', {'Select Date'});
app.EventMarkerLabel1 = uieditfield(app.Country1Tab,
'text', 'Position', [1030 370 120 25], 'Placeholder', 'Event Label');
app.AddMarkerButton1 = uibutton(app.Country1Tab,
'push', 'Position', [1160 370 80 25], 'Text', 'Add Marker',
'ButtonPushedFcn', @(src,evt)addEventMarker(app,1));
app.RemoveMarkerButton1 =
uibutton(app.Country1Tab, 'push', 'Position', [1160 320 80 25],
'Text', 'Remove Marker', 'ButtonPushedFcn',
@(src,evt)removeEventMarker(app,1));

% Create Country2Tab
app.Country2Tab = uitab(app.TabGroup);
app.Country2Tab.Title = 'Country 2 Analysis';

```

```

% Create Axes2_1 (Raw Data)
app.Axes2_1 = uiaxes(app.Country2Tab);
app.Axes2_1.Position = [30 450 400 350];
title(app.Axes2_1, 'Raw Data');
app.Axes2_1.Title.Color = [1 0 0];

% Create Axes2_2 (Model Fit)
app.Axes2_2 = uiaxes(app.Country2Tab);
app.Axes2_2.Position = [450 450 400 350];
title(app.Axes2_2, 'Logistic Model Fit');
app.Axes2_2.Title.Color = [1 0 0];

% Create Axes2_3 (Residuals)
app.Axes2_3 = uiaxes(app.Country2Tab);
app.Axes2_3.Position = [30 80 400 350];
title(app.Axes2_3, 'Residuals Analysis');
app.Axes2_3.Title.Color = [1 0 0];

% Create Axes2_4 (Predictions)
app.Axes2_4 = uiaxes(app.Country2Tab);
app.Axes2_4.Position = [450 80 400 350];
title(app.Axes2_4, 'Future Predictions');
app.Axes2_4.Title.Color = [1 0 0];

% Slider controls for Country 2
% K2 Slider
app.K2SliderLabel = uilabel(app.Country2Tab);
app.K2SliderLabel.Position = [1280 720 200 22];
app.K2SliderLabel.Text = 'K (Carrying Capacity)';
app.K2SliderLabel.FontWeight = 'bold';
app.K2SliderLabel.FontSize = 15;

app.K2Slider = uislider(app.Country2Tab);
app.K2Slider.Position = [900 720 300 3];
app.K2Slider.Limits = [0 10000000];
app.K2Slider.ValueChangedFcn =
createCallbackFcn(app, @K2SliderValueChanged, true);
if app.isDarkMode
    app.K2Slider.FontColor = [1 1 1];
else
    app.K2Slider.FontColor = [0.1 0.1 0.1];
end

app.K2ValueLabel = uilabel(app.Country2Tab);
app.K2ValueLabel.Position = [1220 720 100 22];
app.K2ValueLabel.Text = '0';
app.K2ValueLabel.FontWeight = 'bold';

% r2 Slider
app.r2SliderLabel = uilabel(app.Country2Tab);
app.r2SliderLabel.Position = [1280 650 200 22];
app.r2SliderLabel.Text = 'r (Growth Rate)';
app.r2SliderLabel.FontWeight = 'bold';
app.r2SliderLabel.FontSize = 15;

app.r2Slider = uislider(app.Country2Tab);
app.r2Slider.Position = [900 650 300 3];
app.r2Slider.Limits = [0.001 2.0];
app.r2Slider.ValueChangedFcn =
createCallbackFcn(app, @r2SliderValueChanged, true);
if app.isDarkMode
    app.r2Slider.FontColor = [1 1 1];
else
    app.r2Slider.FontColor = [0.1 0.1 0.1];
end

app.r2ValueLabel = uilabel(app.Country2Tab);
app.r2ValueLabel.Position = [1220 650 100 22];
app.r2ValueLabel.Text = '0.000';
app.r2ValueLabel.FontWeight = 'bold';

% t02 Slider
app.t02SliderLabel = uilabel(app.Country2Tab);
app.t02SliderLabel.Position = [1280 580 200 22];
app.t02SliderLabel.Text = 't0 (Inflection Point)';

app.t02SliderLabel.FontWeight = 'bold';
app.t02SliderLabel.FontSize = 15;

app.t02Slider = uislider(app.Country2Tab);
app.t02Slider.Position = [900 580 300 3];
app.t02Slider.Limits = [1 365];
app.t02Slider.ValueChangedFcn =
createCallbackFcn(app, @t02SliderValueChanged, true);
if app.isDarkMode
    app.t02Slider.FontColor = [1 1 1];
else
    app.t02Slider.FontColor = [0.1 0.1 0.1];
end

app.t02ValueLabel = uilabel(app.Country2Tab);
app.t02ValueLabel.Position = [1220 580 100 22];
app.t02ValueLabel.Text = '1';
app.t02ValueLabel.FontWeight = 'bold';

% Control buttons for Country 2
app.FitModel2Button = uibutton(app.Country2Tab,
'push');
app.FitModel2Button.ButtonPushedFcn =
createCallbackFcn(app, @FitModel2ButtonPushed, true);
app.FitModel2Button.Position = [900 500 140 35];
app.FitModel2Button.Text = 'Auto Fit Model';
app.FitModel2Button.FontWeight = 'bold';

app.ResetSliders2Button = uibutton(app.Country2Tab,
'push');
app.ResetSliders2Button.ButtonPushedFcn =
createCallbackFcn(app, @ResetSliders2ButtonPushed, true);
app.ResetSliders2Button.Position = [1060 500 140 35];
app.ResetSliders2Button.Text = 'Reset Sliders';

% Export button for Country 2
app.Export2Button = uibutton(app.Country2Tab, 'push');
app.Export2Button.ButtonPushedFcn =
createCallbackFcn(app, @Export2ButtonPushed, true);
app.Export2Button.Position = [900 450 200 35];
app.Export2Button.Text = 'Export Results';

% Generate PDF Report button for Country 2
app.Report2Button = uibutton(app.Country2Tab, 'push');
app.Report2Button.ButtonPushedFcn =
createCallbackFcn(app, @GeneratePDFReport2ButtonPushed,
true);
app.Report2Button.Position = [900 400 200 35];
app.Report2Button.Text = 'Generate PDF Report';

% Metrics display for Country 2 (shifted down to avoid
overlap)
app.Metrics2Label = uilabel(app.Country2Tab);
app.Metrics2Label.Position = [900 20 500 280];
app.Metrics2Label.Text = 'Model metrics will appear
here...';
app.Metrics2Label.VerticalAlignment = 'top';
app.Metrics2Label.FontName = 'Courier New';
app.Metrics2Label.FontSize = 10;

% --- Country 2 Event Marker Controls ---
app.EventMarkerDate2 = uidropdown(app.Country2Tab,
'Position', [900 370 120 25], 'Items', {'Select Date'});
app.EventMarkerLabel2 = uieditfield(app.Country2Tab,
'text', 'Position', [1030 370 120 25], 'Placeholder', 'Event Label');
app.AddMarkerButton2 = uibutton(app.Country2Tab,
'push', 'Position', [1160 370 80 25], 'Text', 'Add Marker',
'ButtonPushedFcn', @(src,evt)addEventMarker(app,2));
app.RemoveMarkerButton2 =
uibutton(app.Country2Tab, 'push', 'Position', [1160 320 80 25],
'Text', 'Remove Marker', 'ButtonPushedFcn',
@(src,evt)removeEventMarker(app,2));

% Create ComparisonTab
app.ComparisonTab = uitab(app.TabGroup);

```

```

app.ComparisonTab.Title = 'Comparison Analysis';

% Create ComparisonAxes
app.ComparisonAxes = uiaxes(app.ComparisonTab);
app.ComparisonAxes.Position = [30 300 1000 500];
title(app.ComparisonAxes, 'Country Comparison');
app.ComparisonAxes.Title.Color = [1 0 0];

% Create ExportComparisonButton
app.ExportComparisonButton =
uibutton(app.ComparisonTab, 'push');
app.ExportComparisonButton.ButtonPushedFcn =
createCallbackFcn(app, @ExportComparisonButtonPushed,
true);
app.ExportComparisonButton.Position = [1100 700 250
40];
app.ExportComparisonButton.Text = ' Export
Comparison';
app.ExportComparisonButton.FontSize = 12;
app.ExportComparisonButton.FontWeight = 'bold';

% Create ComparisonMetricsLabel
app.ComparisonMetricsLabel =
uiblabel(app.ComparisonTab);
app.ComparisonMetricsLabel.Position = [1100 300 400
350];
app.ComparisonMetricsLabel.Text = 'Comparison
metrics will appear here...';
app.ComparisonMetricsLabel.VerticalAlignment = 'top';
app.ComparisonMetricsLabel.FontName = 'Courier
New';
app.ComparisonMetricsLabel.FontSize = 11;

% Create StatusLabel
app.StatusLabel = uilabel(app.UIFigure);
app.StatusLabel.Position = [20 40 1400 35];
app.StatusLabel.FontSize = 12;
app.StatusLabel.FontWeight = 'bold';
app.StatusLabel.Text = 'Status: Ready - Upload your
COVID-19 data file to begin';

% ---- Advanced Analytics Tab ----
app.AdvancedTab = uitab(app.TabGroup, 'Title', ' 📊
Advanced Analytics');
app.AdvAnalysisDropdown =
uidropdown(app.AdvancedTab, 'Position', [30 770 200 22], ...
'Items', {'Country Dashboard','World Dashboard','Date
Statistics','Compare Countries','Saturation Dates','Metrics
Only'});
app.AdvCountry1Dropdown =
uidropdown(app.AdvancedTab, 'Position', [240 770 180 22], ...
'Items', [{'Select Country'}, app.CountryList]);
% --- FIX: Assign callbacks to the correct properties ---
app.AdvCountry1Dropdown.Editable = true;

app.AdvCountry2Dropdown =
uidropdown(app.AdvancedTab, 'Position', [430 770 180 22], ...
'Items', [{'Select Country'}, app.CountryList]);
% --- FIX: Assign callbacks to the correct properties ---
app.AdvCountry2Dropdown.Editable = true;

app.AdvDateField = uieditfield(app.AdvancedTab, 'text',
'Position', [620 770 130 22], ...
'Placeholder', 'YYYY-MM-DD');
app.AdvRunButton = uibutton(app.AdvancedTab, 'push',
'Position', [770 770 140 40], ...
'Text', ' 🚀 Run Analysis', 'ButtonPushedFcn',
@(src,evt)runAdvAnalysis(app));
app.AdvRunButton.FontSize = 18;
app.AdvMetricsButton = uibutton(app.AdvancedTab,
'push', 'Position', [930 770 140 40], ...
'Text', 'Show Metrics', 'ButtonPushedFcn',
@(src,evt)showAdvMetrics(app));
app.AdvMetricsButton.FontSize = 18;
% Create axes for Advanced Analytics tab

```

```

app.AdvAxes1 = uiaxes(app.AdvancedTab, 'Position',
[40 410 350 300]);
app.AdvAxes2 = uiaxes(app.AdvancedTab, 'Position',
[430 410 350 300]);
app.AdvAxes3 = uiaxes(app.AdvancedTab, 'Position',
[40 70 350 300]);
app.AdvAxes4 = uiaxes(app.AdvancedTab, 'Position',
[430 70 350 300]);
% Create AdvResultLabel for metrics/statistics display
app.AdvResultLabel = uilabel(app.AdvancedTab);
app.AdvResultLabel.Position = [850 410 650 350];
app.AdvResultLabel.Text = 'Results will appear here...';
app.AdvResultLabel.FontSize = 26;
app.AdvResultLabel.VerticalAlignment = 'top';
app.AdvResultLabel.FontName = 'Courier New';
if app.isDarkMode
app.AdvResultLabel.FontColor = [1 1 1];
else
app.AdvResultLabel.FontColor = [0.1 0.1 0.1];
end
% Set event marker controls background color for theme
if app.isDarkMode
markerBG = [0.20 0.23 0.28];
markerFG = [0.95 0.97 1];
else
markerBG = [1 1 1];
markerFG = [0.1 0.1 0.1];
end
% Country 1
app.EventMarkerDate1.BackgroundColor = markerBG;
app.EventMarkerDate1.FontColor = markerFG;
app.EventMarkerLabel1.BackgroundColor = markerBG;
app.EventMarkerLabel1.FontColor = markerFG;
app.AddMarkerButton1.BackgroundColor = markerBG;
app.AddMarkerButton1.FontColor = markerFG;
app.RemoveMarkerButton1.BackgroundColor =
markerBG;
app.RemoveMarkerButton1.FontColor = markerFG;
% Country 2
app.EventMarkerDate2.BackgroundColor = markerBG;
app.EventMarkerDate2.FontColor = markerFG;
app.EventMarkerLabel2.BackgroundColor = markerBG;
app.EventMarkerLabel2.FontColor = markerFG;
app.AddMarkerButton2.BackgroundColor = markerBG;
app.AddMarkerButton2.FontColor = markerFG;
app.RemoveMarkerButton2.BackgroundColor =
markerBG;
app.RemoveMarkerButton2.FontColor = markerFG;

% Show the figure after all components are created
app.UIFigure.Visible = 'on';
end
end
% App initialization and construction
methods (Access = public)

% Construct app
function app = COVID19_Logistic_GUI_App
% Create UIFigure and components
createComponents(app)

% Register the app with App Designer
registerApp(app, app.UIFigure)

% Execute the startup function
runStartupFcn(app, @startupFcn)
end
% Code that executes before app deletion
function delete(app)
% Delete UIFigure when app is deleted
delete(app.UIFigure)
end
end
end

```


4. Implementation:

4.1 Description:

The implementation of this project combines mathematical modeling, data analysis, and GUI development using MATLAB. The primary objective was to provide users with an intuitive, interactive platform to explore the spread of COVID-19 using a logistic growth model. The application was built using **MATLAB App Designer**, which allows for creating a user-friendly graphical interface with integrated code logic and visual components.

At the core of the implementation is the **logistic growth function**, which models the cumulative number of COVID-19 cases over time. The application reads a .csv dataset(<https://catalog.ourworldindata.org/garden/covid/latest/compact/compact.csv> from Our World in Data) containing country-wise, time-series COVID-19 data, including date, location, and cumulative case counts. Users can select one or two countries for analysis through dropdown menus. Once a country is selected, the application extracts the corresponding data, processes it, and applies the **nonlinear curve fitting method** using MATLAB's *lsqcurvefit* function. This process estimates three key parameters: carrying capacity (K), growth rate (r), and inflection point (t_0).

The GUI is divided into multiple tabs for better organization and functionality. These tabs include features for:

- Displaying the original data and fitted curve,
- Viewing residuals between actual and predicted values,
- Forecasting future trends using extended time simulations,
- Comparing the spread between two countries side-by-side, and
- Performing advanced analytics such as parameter sensitivity and saturation date estimation.

The app also includes robust error handling for invalid inputs or missing data. It automatically updates plots and statistics when new selections are made, ensuring real-time interaction. Colour themes adapt based on user mode (e.g., dark/light), improving accessibility.

This implementation provides a complete and self-contained environment for exploring logistic model-based COVID-19 forecasts, designed to be both educational and functional for users with varying technical backgrounds.

5. Design Analysis and Evaluation:

5.1 Design Consideration:

In designing the COVID-19 spread modeling tool, several key considerations were taken into account to ensure the system would be accurate, user-friendly, scalable, and adaptable to various real-world data formats. These considerations guided both the mathematical modeling strategy and the implementation of the MATLAB-based GUI.

1. Choice of Mathematical Model

The logistic growth model was selected due to its simplicity and suitability for modeling single-wave epidemic curves. It balances interpretability with effectiveness, using three intuitive parameters—carrying capacity (K), growth rate (r), and inflection point (t_0). This model avoids overfitting while still capturing the key dynamics of cumulative case growth.

2. Real-World Data Compatibility

The tool is designed to handle real-world datasets sourced from repositories like *Our World in Data*. These datasets often contain missing values, inconsistent formatting, or outliers. Data cleaning processes, such as interpolation and filtering, were integrated to enhance robustness.

3. User Interface and Accessibility

A core design goal was to make the application intuitive and accessible to users with varying levels of technical expertise. The GUI allows for:

- Easy file uploading
- Country selection and comparison
- Visual exploration of data and forecasts
- Exporting results for external analysis

Dropdowns, sliders, plots, and feedback labels were chosen deliberately to guide the user experience.

4. Parameter Estimation Stability

Logistic model fitting requires optimization, which can be sensitive to initial guesses and bounds. The design includes pre-defined fitting ranges and fallback mechanisms (e.g., using *fminsearch* if *lsqcurvefit* fails) to ensure stability and prevent convergence issues.

5. Visual Interpretation and Diagnostics

To help users assess model reliability, the tool provides:

- Residual plots
- Forecast projections
- Model fit diagnostics (RMSE and R^2)

These enable quick identification of poor fits or anomalies in the data.

6. Scalability and Modifiability

While this project focused on single-wave logistic modeling, the system was designed with modularity in mind. This allows future upgrades such as:

- Multi-wave models (e.g., piecewise logistics or SEIR)
- Inclusion of policy/event markers
- Real-time data integration via APIs

5.2 Investigations:

5.2.1 Design of Experiment:

The experiment was designed to validate the effectiveness of the logistic growth model in modeling COVID-19 cumulative case data. It aimed to determine how well the model fits real-world data across different countries and how accurately it can forecast future trends.

The experimental design followed these steps:

- **Country Selection:** A diverse set of countries was selected based on outbreak size, region, and pandemic phase to ensure varied epidemic patterns.
- **Data Time Frame:** For each country, data was collected from the date of the first reported case up to a consistent cutoff date.
- **Fitting Strategy:** The logistic model was fit using non-linear least squares (`lsqcurvefit`) in MATLAB, with fallback to `fminsearch` for robustness.
- **Evaluation Metrics:** RMSE and R^2 were used to assess the quality of fit. Residual plots and forecast curves were used to validate the visual consistency of the model.

5.2.2 Data Collection:

Data for the experiment was sourced from the **Our World in Data** COVID-19 dataset, which provides cumulative case numbers by date and country.

Steps:

- The dataset was loaded as a .csv file within the MATLAB app.
- Users selected countries using dropdowns in the GUI.
- For each country, the app extracted relevant fields: date and total cases.
- Dates were converted to “days since first case” for model fitting.
- Any missing, non-numeric, or inconsistent values were handled through interpolation or exclusion to maintain data quality.

5.2.3 Results:

After fitting the logistic growth model to selected countries using the custom-built MATLAB GUI, the following trends and insights were observed:

Quantitative Observations:

- **Good Fit in Single-Wave Cases:** Countries like China and New Zealand, with single-peak outbreaks, showed strong logistic characteristics with high R^2 values (often > 0.98) and low RMSE.
- **Poor Fit in Complex Outbreaks:** Countries experiencing multiple waves, such as the USA or India, demonstrated deviation from the logistic model due to secondary outbreaks, policy changes, or underreporting.

Parameter Behavior:

- **Carrying Capacity (K):** Proportional to population size and testing capacity.
- **Growth Rate (r):** Reflected infection speed; higher in densely populated areas.
- **Inflection Point (t_0):** Indicated peak spread; varied depending on response timing.

Forecasting Accuracy:

The logistic model could reasonably forecast the point of saturation for countries nearing the end of a single wave. However, its predictions diverged in ongoing or multi-wave cases.

GUI Output & Visual Results:

The MATLAB GUI was used to visualize:

- **Model Fit:** Comparison of actual vs. predicted cumulative cases.
- **Residuals Plot:** Deviations between predicted and actual data.

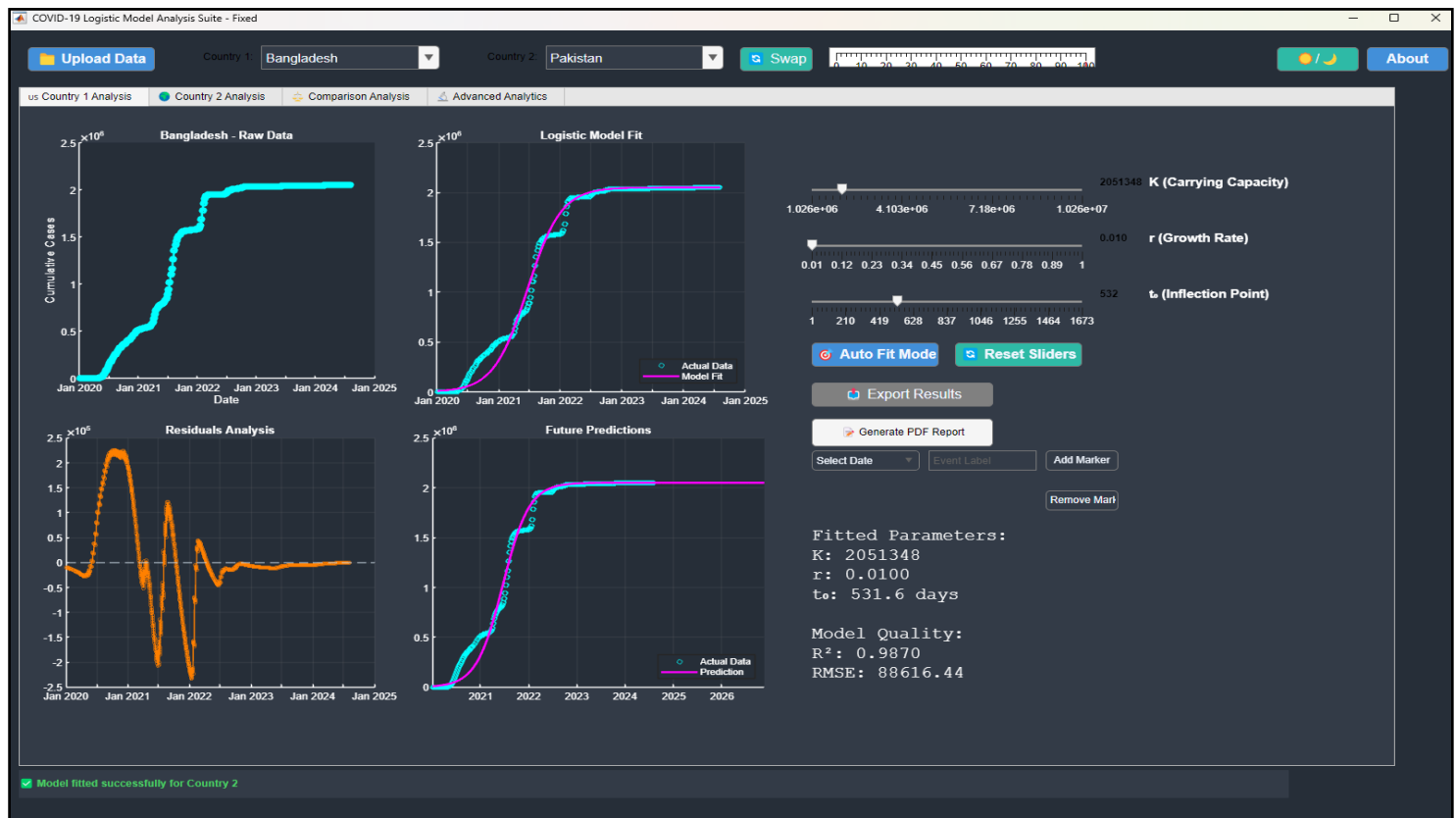


Figure 5.1: GUI Output – Model Fit, Forecast and Residual for Country 1

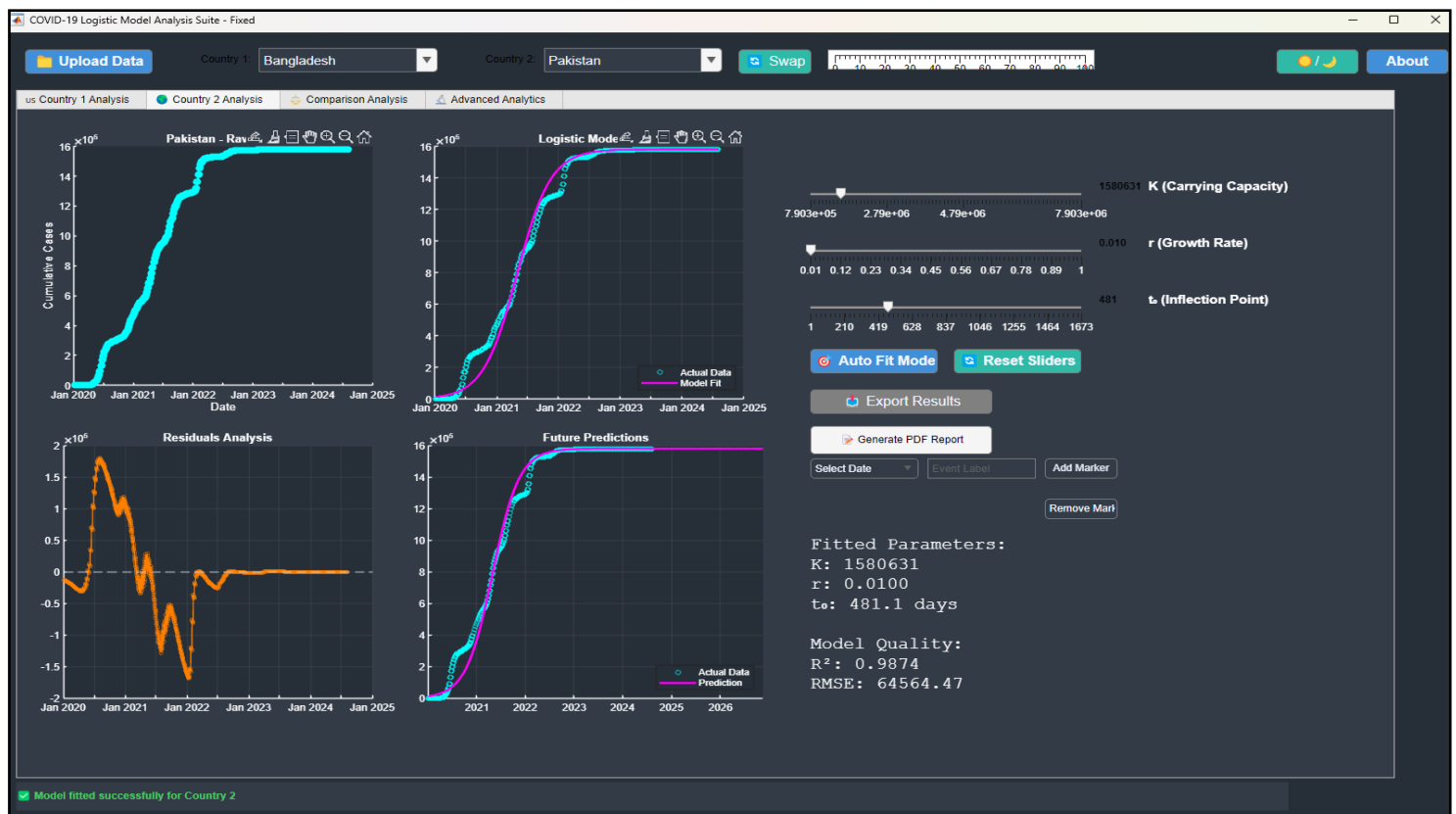


Figure 5.2: GUI Output – Model Fit, Forecast and Residual for Country 2

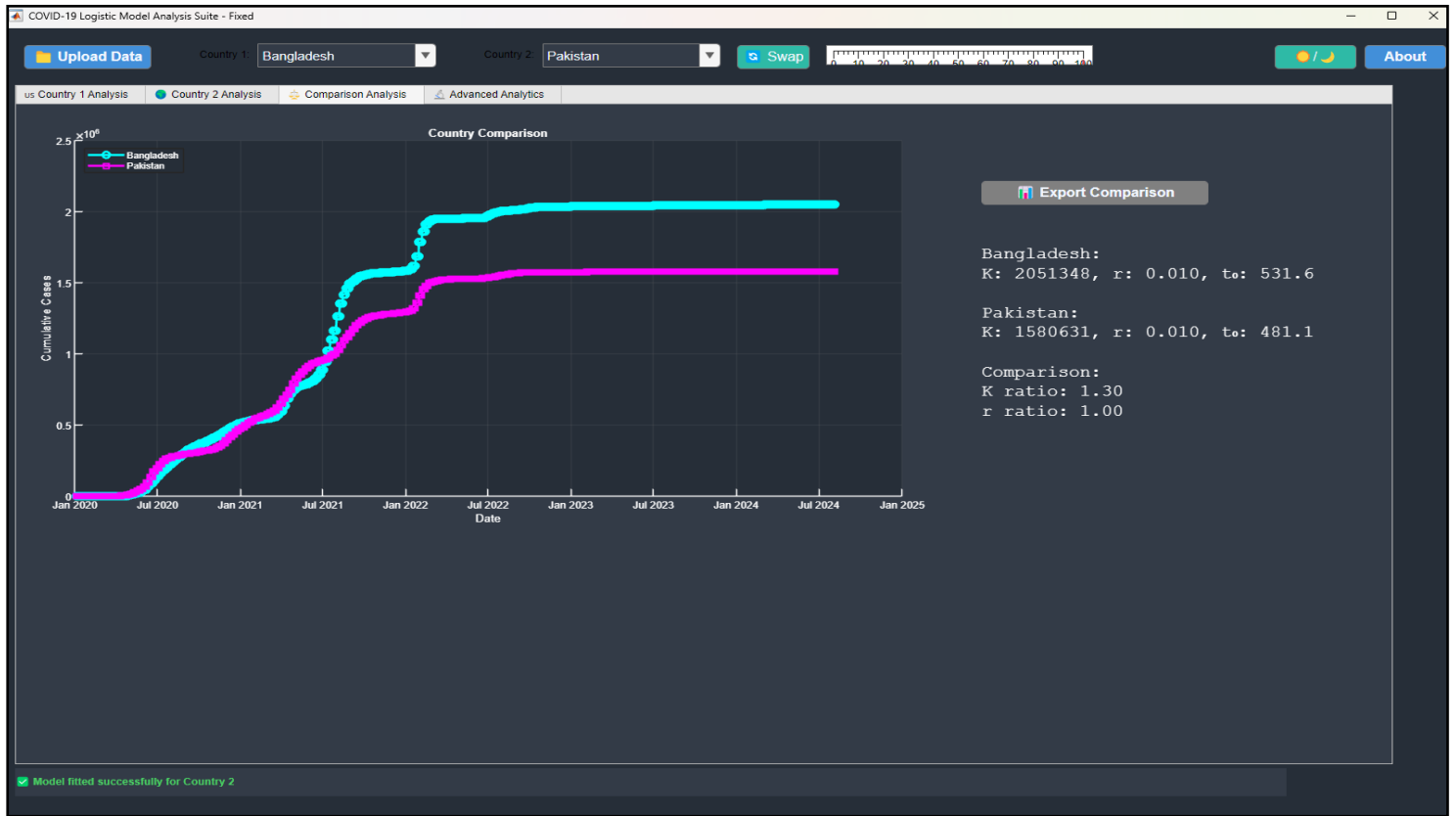


Figure 5.3: GUI Output – Country Comparison Plot

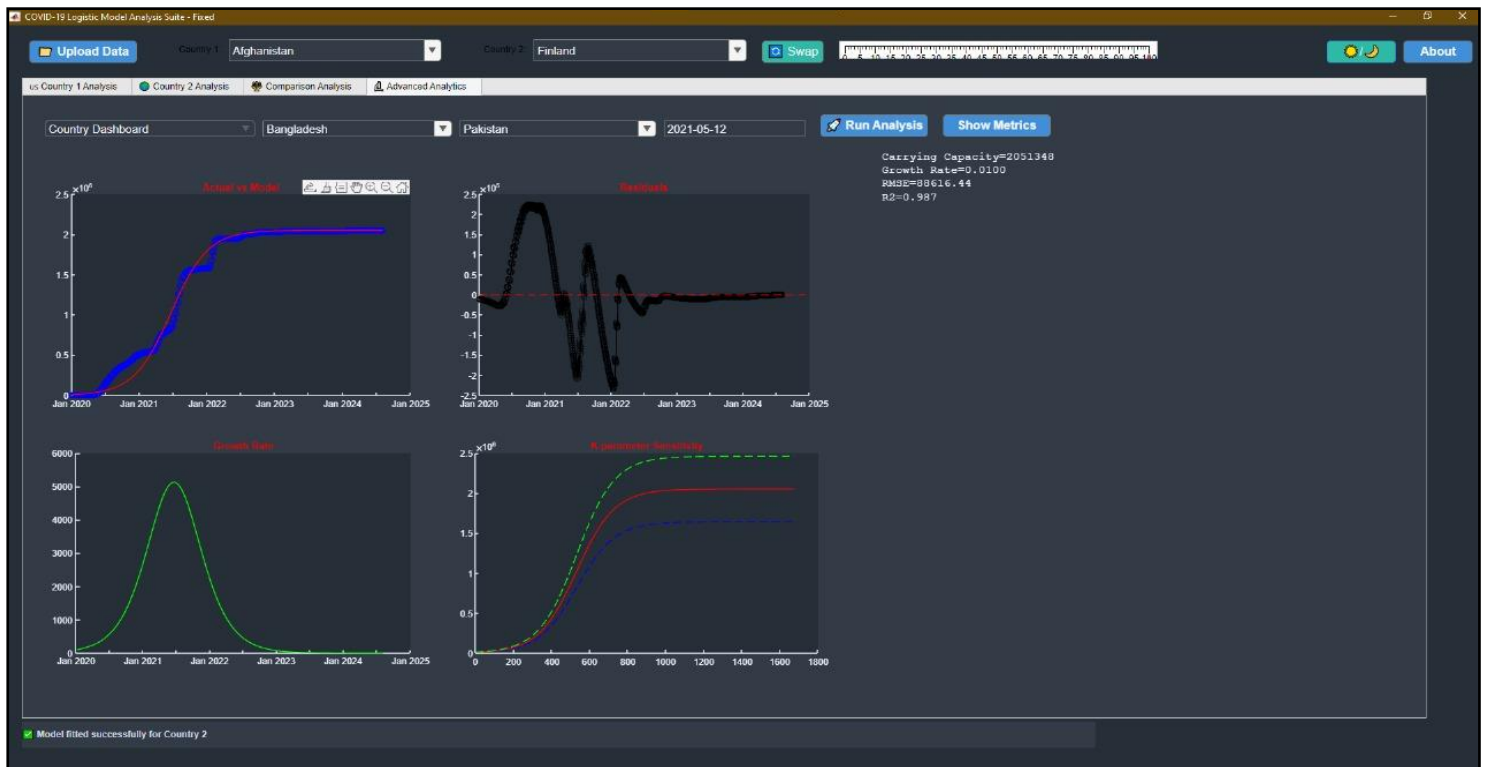


Figure 5.4: GUI Output – Advanced Analysis

5.2.4 Conclusion on Data:

The logistic growth model provided a reasonable approximation for the spread of COVID-19 in countries with a single dominant wave. However, its effectiveness was limited in:

- Countries with multi-phase outbreaks
- Data with inconsistent reporting
- Scenarios involving strong policy interventions

This highlights the model's usefulness as a baseline forecasting tool, but also its limitations. For higher accuracy, more complex models (e.g., SEIR, multi-phase logistic) or hybrid approaches may be required in future work.

5.3 Limitation of Tools:

Despite successful implementation, the project faced several limitations:

1. Model Constraints

- The logistic model assumes a single-wave outbreak and doesn't handle multiple peaks or external interventions like lockdowns.

2. Data Issues

- Real-world data often had missing, inconsistent, or delayed entries.
- Negative or zero values required interpolation, possibly introducing bias.

3. GUI Limitations

- With large datasets, the GUI could lag.
- Poor initial parameter guesses sometimes caused optimization to fail.
- Manual slider tuning could lead to overfitting if used without care.

4. Computational Efficiency

- Fitting algorithms were time-consuming on large datasets.
- Lack of parallelization in App Designer affected performance.

Even with these constraints, the tool effectively demonstrates interactive epidemic modeling using MATLAB

5.4 Sustainability Evaluation:

The project was built with long-term usability and adaptability in mind:

1. Maintainability & Reusability

- Modular, well-commented code allows for easy updates.
- The tool can be adapted for future epidemics by changing the dataset.

2. Accessibility

- Designed for users with minimal MATLAB knowledge.
- Supports dark/light themes for better visual comfort.

3. Extendibility

- Can be upgraded to support complex models (e.g., SEIR) with little structural change.
- Compatible with various file formats like .csv, .xlsx, and .mat.

4. Educational Value

- Useful for teaching numerical modeling and data analysis through hands-on interaction.

Overall, the project offers a solid, sustainable foundation for further development and academic use.

6. Reflection on Individual and Team Work:

6.1 Individual Contribution of Each Member:

All four team members contributed equally to the completion of this project, with responsibilities assigned based on individual strengths and interests.

Sasshata Talukder-(2206136)

- Conducted background research on logistic models and epidemic dynamics.
- Assisted in data preprocessing, outlier removal, and analysis.
- Helped design the initial structure of the GUI and input handling logic.

S. M. Sindid Islam-(2206147)

- Led GUI development using MATLAB App Designer.

- Built key interactive components including dropdowns, tabs, sliders, and visualization panels.
- Implemented country comparison and export functionalities with full user interactivity.

Rajib Khan-(2206152)

- Handled logistic model fitting using *lsqcurvefit* and fallback methods.
- Conducted testing across different countries and tuned initial parameter guesses.
- Worked on residual analysis, forecasting, and model evaluation metrics.

Iftekhhar-E-Islam-(2206153)

- Focused on error handling, UI theme customization, and progress indicators.
- Drafted and edited major report sections including design analysis and sustainability.
- Managed group coordination, version control, and final documentation formatting.

Each member reviewed and tested the full system to ensure performance and consistency. The final output is a result of joint planning and collaboration.

6.2 Log Book of Project Implementation:

Day	Activity	Team Members Involved
Day 1	Finalized project topic and discussed roles and objectives	All
Day 2	Researched logistic growth models and epidemic modeling techniques	Sasshata, Iftekhhar
Day 3	Collected and reviewed COVID-19 datasets from <i>*Our World in Data*</i>	Sasshata, Sindid
Day 4	Implemented basic logistic model and tested with example data	Rajib, Sasshata

Day 5	Created initial GUI structure using App Designer	Sindid, Iftekhar
Day 6	Developed dropdowns, tabs, and dynamic interface elements	Sindid
Day 7	Integrated <code>'lsqcurvefit'</code> and tested parameter estimation	Rajib
Day 8	Developed plots for model fit and residual analysis	Rajib, Sasshata
Day 9	Implemented sliders, reset buttons, and forecast plotting in GUI	Sindid, Iftekhar
Day 10	Added country comparison feature and export functionality	Sindid, Rajib
Day 11	Improved theme switching, error prompts, and user feedback elements	Iftekhar
Day 12	Export feature testing and final data table formatting for output	Sindid, Rajib
Day 13	Full application testing and debugging across different countries	All
Day 14	Drafted sections: Abstract, Design Method, Results, and GUI overview	Iftekhar, Sasshata
Day 15	Final review, formatting, and report submission	All

7. Communication to External Stakeholders:

7.1 Executive Summary:

This project presents a MATLAB-based simulation tool that models the cumulative spread of COVID-19 using a logistic growth function. Developed as part of the EEE 212 Numerical Technique Laboratory course, the tool combines mathematical modeling with interactive visualization to analyze epidemic trends using real-world data.

The core objective was to implement a logistic model capable of estimating key parameters such as carrying capacity (K), growth rate (r), and inflection point (t_0) from country-wise COVID-19 case data. The application features a user-friendly graphical user interface (GUI), built with MATLAB App Designer, allowing users to upload datasets, fit models, visualize residuals, forecast future cases, and compare two countries side by side.

Key features include:

- Support for [.csv](#), [.xlsx](#), and [.mat](#) data formats
- Parameter fitting via [lsqcurvefit](#) with fallback options
- Residual and forecast plots for model evaluation
- Exportable results and customizable visual themes

The tool demonstrates high usability and adaptability, particularly for single-wave epidemic modeling. While the logistic model has limitations in multi-wave scenarios, the framework lays a strong foundation for future enhancements such as SEIR integration, real-time data updates, and extended analytics.

This project not only showcases the application of numerical techniques in real-world scenarios but also highlights the value of combining algorithmic modeling with intuitive software design.

7.2 User Manual:

This MATLAB app enables users to analyze COVID-19 case data using logistic growth models. Users can visualize, compare, and export infection curves and predictions based on cumulative case data.

To get started, user have to upload a [.csv](#) file. An informative [.csv](#) file is in [Our World in Data](#) website. User is able to switch between light and dark mode from the right top light/dark swap button.

Country 1 analysis

User can choose/write country name on the country 1 dropdown box and selected the desired one. Then clicking Auto Fit Mode will lead to four plots of Raw Data, Logistic model, Residuals analysis, Future predictions.

Using the sliders of K r t_0 and varying these parameters will show results on the plots. The Export result button will generate a .csv file of that country raw data and generate PDF report will help to convert the plots into a PDF file and a .txt file with the parameters with other info like RMSE R^2 .

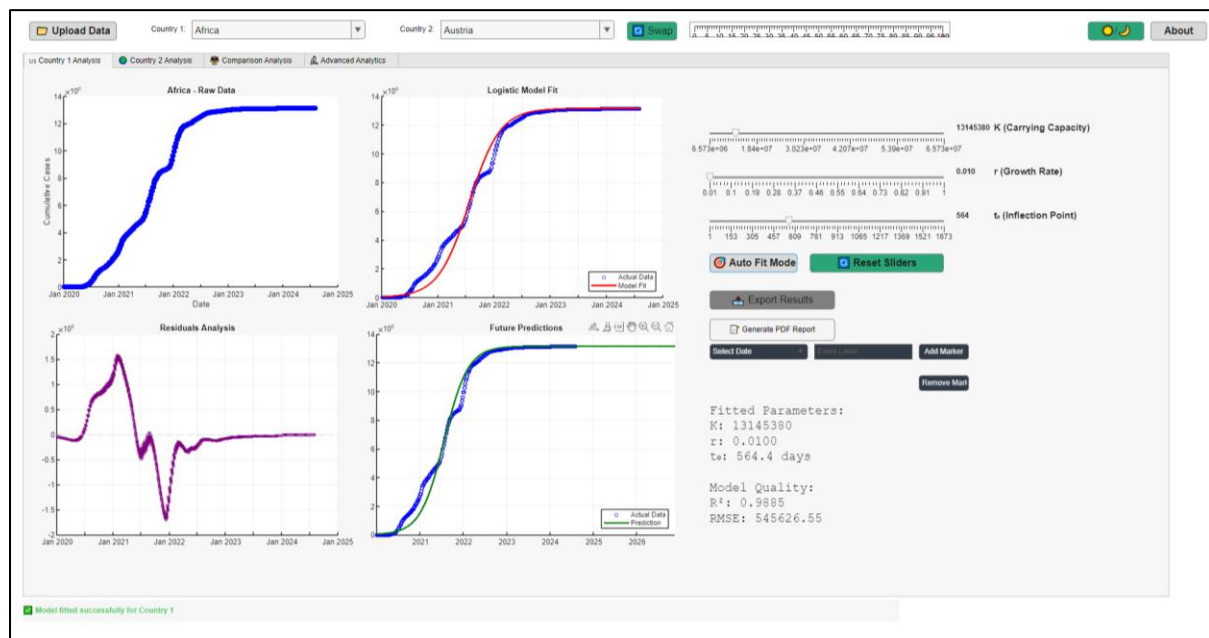


Figure 1: - Country 1 Analysis GUI

Then select date lets user to choose a date and label it with any name of interest. The add marker button lets user to mark the date with label on the Raw data plot. User can mark and label dates many times. To remove the mark label, just simply clicking Remove marker will do that.

Country 2 analysis

This section is same as country 1 analysis, but user can see info about another country without clearing country 1. Just simply choosing country from the dropdown box and selecting as like previous section will do the job.

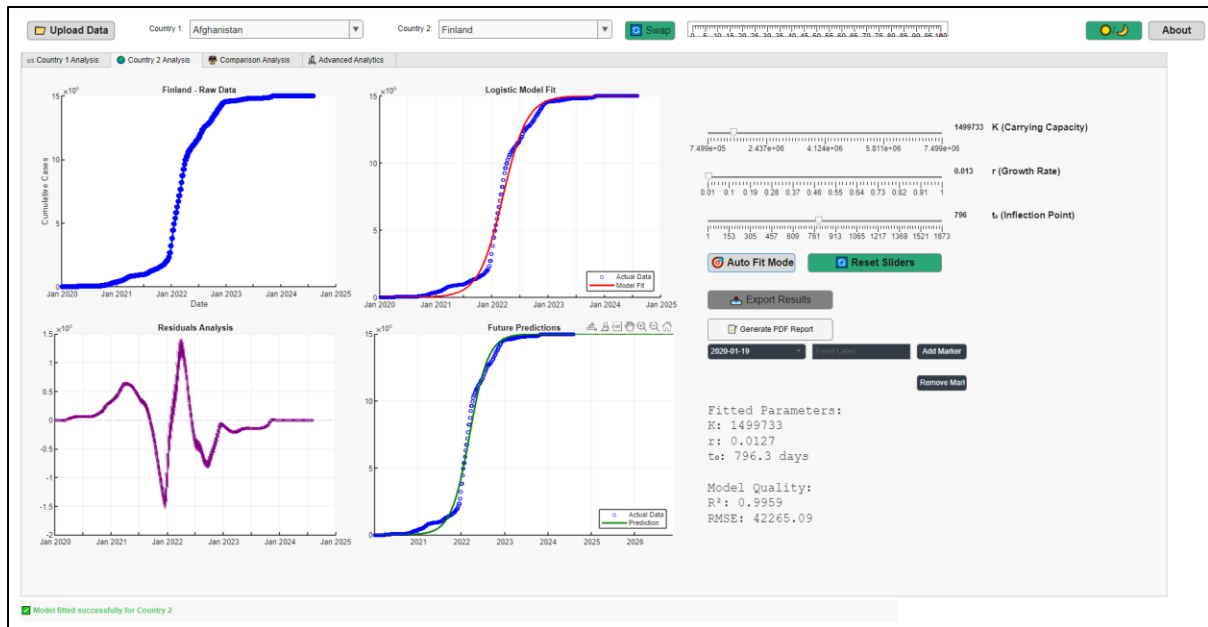
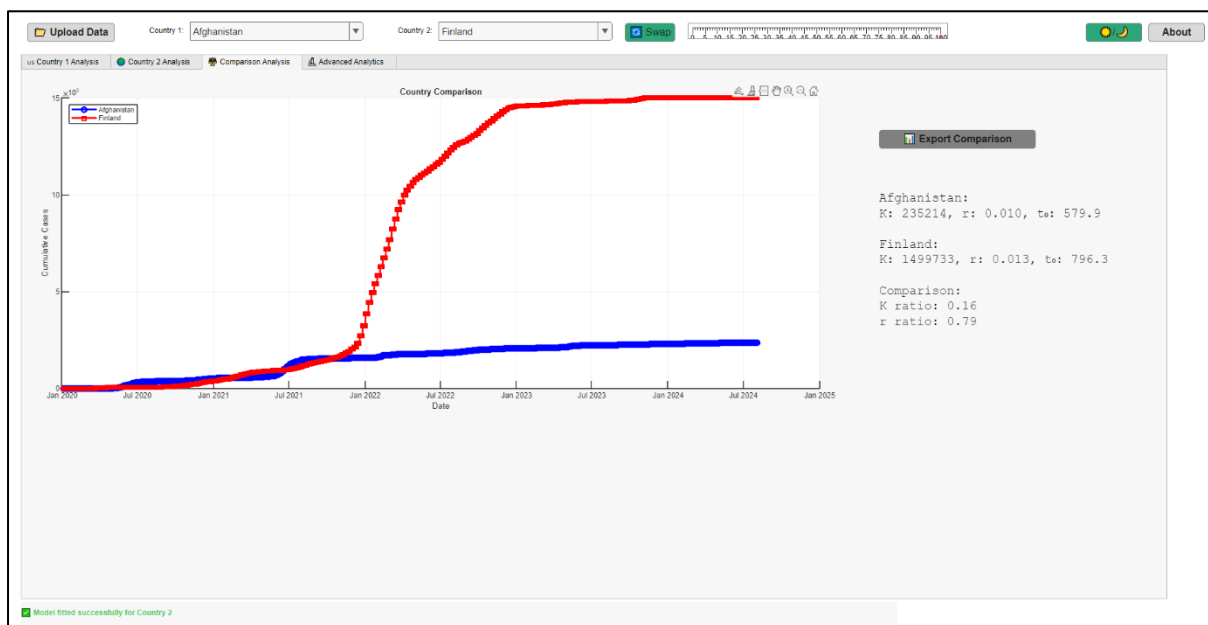


Figure 2: - Country 2 Analysis GUI

Also, user can switch between two countries by the swap button.

Country Comparison

This section is linked with previous sections. The section will display a comparison between Country 1 and Country 2 (user have already selected it in the previous sections) in a single plot.



User can also export the comparison info by Export comparison button.

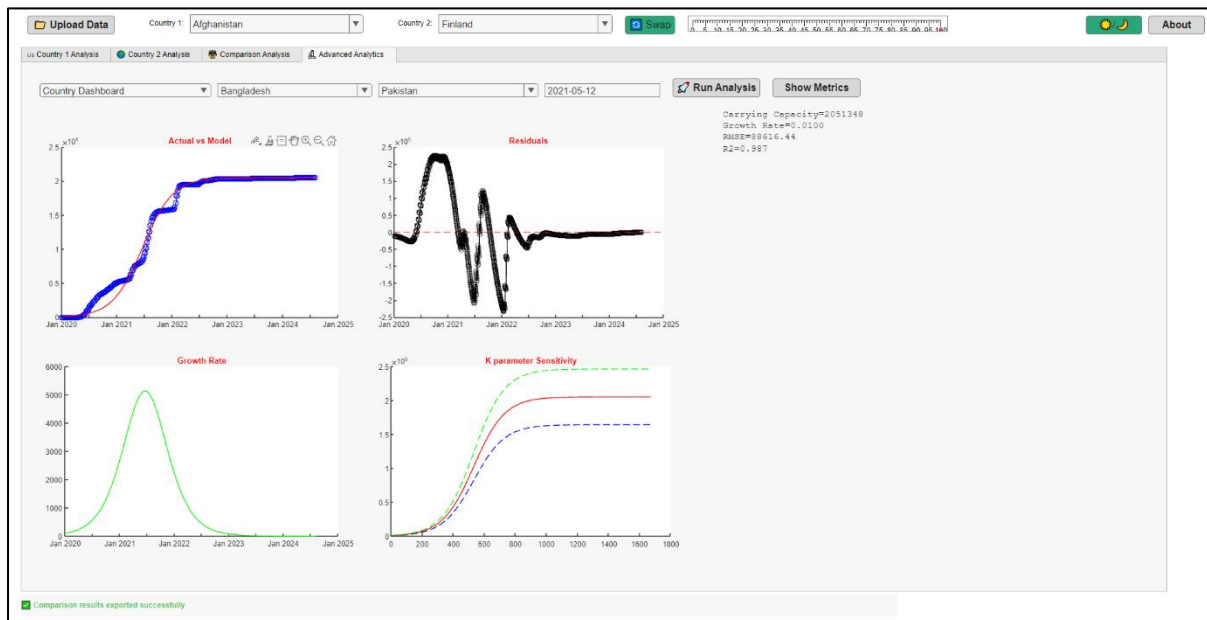
Advanced Analytics

This is the last segment of the GUI. User can get some additional info about the model.

The first option is country Dashboard. User can see the growth rate and K parameter sensitivity plots.

The max growth rate can also be detected from this segment.

Then there is world dashboard where it shows data for the whole world. There will be parameters like carrying capacity, avg Growth rate, Root mean square error (RMSE) and R2.



In 3rd, Date statistics, user can write a date and get to know about the total infected and total death numbers. User must input date as YYYY-MM-DD. Then there is compare country's part to compare. Lastly, the saturation date part that shows the date of saturation (the part when cumulative cases become almost constant).

7.3 Youtube Link:

The final presentation for this project can be viewed at the following link:
<https://youtu.be/kMos3fnVl4Q>

8. Project Management and Cost Analysis:

The development of this project incurred no direct financial cost. MATLAB and its App Designer tool were used for both modelling and GUI development, and these were made available through our university's institutional license. As a result, we did not need to purchase any additional software or resources. All datasets used were publicly available and free to access. Therefore, the total cost for implementing this project was effectively zero, thanks to institutional support and the use of open data.

9. Future Work:

9.1 Model Stability:

Model stability refers to the consistency and reliability of the logistic growth model when applied to different datasets or under slight changes in input parameters. In the context of this project, model stability is critical to ensure that the predicted COVID-19 case trends remain meaningful and robust despite the variability in real-world data.

The logistic model used in this application is mathematically stable in structure; however, its practical stability depends heavily on the quality and completeness of the input data. For example, sudden spikes, underreporting, or changes in testing policies can affect the smoothness of the cumulative case curve and influence the accuracy of the fitted parameters. To mitigate this, the application includes basic preprocessing steps such as interpolation for missing values and exclusion of obvious outliers.

Stability is also influenced by the initial guesses and bounds provided to the curve-fitting algorithm (*lsqcurvefit*). Poor initial values can lead to slow convergence or local minima. To address this, the model uses reasonable, empirically chosen starting points and bounded optimization to ensure consistent fitting across countries.

Furthermore, one of the Advanced Analytics plots in the application specifically visualizes the sensitivity of the model to changes in the carrying capacity (K). This helps assess how small changes in parameters affect the model's output. A model is considered more stable if minor variations in input do not lead to significant distortions in the prediction curve.

In future enhancements, model stability can be improved through techniques like rolling-window fitting, multi-wave modelling, or adaptive parameter updating based on real-time data. These would allow the model to better adapt to ongoing changes in the epidemic curve, especially in regions experiencing multiple waves or irregular data reporting.

Overall, the current implementation demonstrates acceptable model stability for single-wave scenarios, while providing a strong foundation for further development and robustness improvement.

9.2 Model Accuracy:

The accuracy of the logistic growth model in this project depends on several factors related to data quality, fitting techniques, and model assumptions. Below are key considerations that influence the accuracy of predictions:

- Accurate and consistent COVID-19 case data is essential.

- Underreporting, delayed updates, or abrupt changes in reporting standards can reduce model accuracy.
- The logistic model assumes a single outbreak wave.
- In countries with multiple waves or prolonged outbreaks, this can lead to overfitting or underfitting.
- The use of *lsqcurvefit* for nonlinear optimization provides good parameter fitting, but the result depends on initial guesses and bounds.
- Poor initialization can lead to convergence on local minima.
- Residual plots help assess the quality of fit.
- Randomly scattered residuals indicate a good model; visible trends or patterns suggest model mismatch.
- Short-term forecasts tend to be more reliable than long-term predictions, as they are less affected by sudden policy or behavioural changes.
- Metrics such as **R² (coefficient of determination)** and **RMSE (root mean squared error)** are used to quantify model fit quality.
- Higher R² and lower RMSE indicate better accuracy.

This analysis helps in understanding the limitations of the model's predictive ability and sets the groundwork for improving future accuracy by incorporating more complex models or real-time data adjustments.

10. References:

10.1 Ref from Websites

1. <https://catalog.ourworldindata.org/garden/covid/latest/compact/compact.csv>
2. https://www.mdpi.com/idr/idr-13-00046/article_deploy/html/images/idr-13-00046-g001.png